

From Modern Deep Reinforcement Learning to Continual Reinforcement Learning

Ying Wen

Shanghai Jiao Tong University
Shanghai Innovation Institute

ying.wen@sjtu.edu.cn

August 26, 2026

Abstract

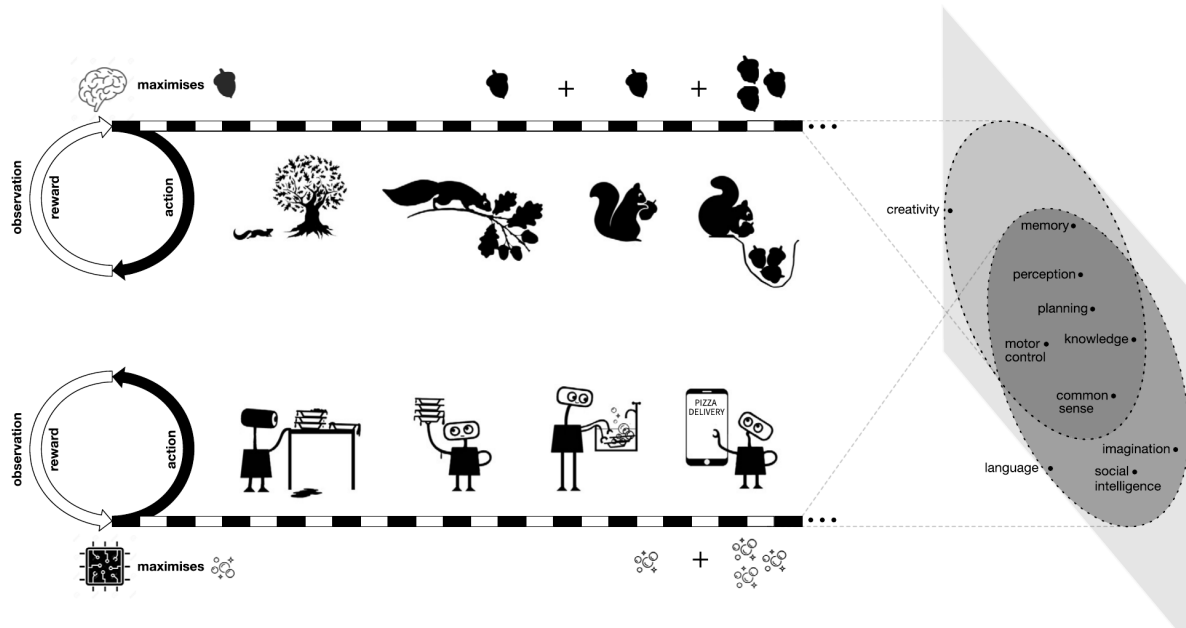
Modern deep reinforcement learning has established effective methods for high-dimensional prediction and control, usually within designer-specified environments. Continual reinforcement learning asks how a bounded agent can form and revise the objects that those environments ordinarily supply. This perspective organizes that problem around direct interaction with a big world and the dependency structure of the Alberta Plan: agent-state construction, general value prediction, continuing control, transition models and temporal abstraction, autonomous learning objectives, and multi-timescale meta-learning. It connects classical reinforcement-learning abstractions to contemporary language-agent and embodied settings while separating research hypotheses, component-level evidence, and integrated continual-agent claims. The resulting agenda evaluates long-run adaptation, retention, plasticity, recovery, and resource use within a continuing stream of experience.

Keywords: reinforcement-learning, deep-reinforcement-learning, continual-reinforcement-learning, agent-state, world-models, embodied-ai

In autumn, a squirrel searches for food among oak trees. Hunger does not decompose the world into separate assignments called “nut recognition,” “route planning,” “grasp control,” and “location memory”; it continually affects the squirrel through the consequences of action. To keep obtaining food in the future, the squirrel must identify edible nuts, traverse branches and manipulate them, choose cache locations, remember where food was buried, and may even use deceptive caching movements to mislead potential thieves. **Reward Is Enough** uses this example to argue that perception, knowledge, motor control, planning, memory, and social intelligence can arise together as capabilities required for long-run reward maximization.[1]

The same paper imagines a kitchen robot whose reward signal measures cleanliness. The robot does not face a fixed list of “dirty objects”: utensils may be occluded, stained in unfamiliar ways, or misplaced; a child’s remark may predict the next mess; and each cleaning action changes the conditions for later action. To keep the kitchen clean over time, the robot must distinguish clean from dirty utensils, learn how to manipulate them, remember where they tend to appear, anticipate future mess from dialogue, and coordinate with the people who use the kitchen. Cleanliness supplies a unified evaluative consequence, but effective behavior depends on an interconnected set of capabilities that develops through experience.

These examples state a research hypothesis, not a proved general theorem. A scalar reward does not automatically produce intelligence, and specifying reward does not replace state construction, credit assignment, exploration, prediction, control, or learning mechanisms. The scientific problem is to explain how a capacity-limited agent, acting within one continuing stream of experience in a complex world, makes these capabilities serve long-run outcomes and continues to revise them as the world changes. Openmind Research Institute poses a closely related



Opening figure | Top: a squirrel obtains food through an observation–action–reward loop. Bottom: a kitchen robot improves cleanliness through the same kind of loop. The projection on the right represents capabilities—such as perception, memory, planning, knowledge, and language—that may form and become expressed in these streams of experience. Original figure from David Silver, Satinder Singh, Doina Precup, and Richard S. Sutton, *Reward Is Enough* [1], Figure 1; reproduced under [CC BY-NC-ND 4.0](#).

question: how minds can form from computation interacting with a general continuing data stream, and how they can learn in real time from sensorimotor experience.[2]

Reinforcement learning originally set out to study precisely this closed loop: an agent acts and acquires experience; experience changes the agent; the changed agent then produces new actions and new experience. Reinforcement learning is therefore, by its nature, an interactive paradigm for continual learning.

The learning objective of reinforcement learning should be to learn, through continual interaction and from first-person experience, an agent state that supports action; General Value Functions (GVFs) whose predictions can be tested by future experience; control policies that improve long-run outcomes; transition models that support temporal abstraction and planning; and multi-timescale learning mechanisms that regulate all of the above. Intelligence forms as these internal objects are constructed, used, tested, and revised through experience—not merely when a policy is computed after states and rewards have already been specified.

The basic research framework of this essay is the **Alberta Plan for AI Research**, proposed by Richard Sutton, Michael Bowling, and Patrick Pilarski. The Alberta Plan studies computational agents that interact over long periods with worlds far more complex than themselves and that continually learn prediction and control through action. It advances four central commitments: learning from ordinary observation–action–reward experience; integrating perception, action, learning, and planning on a common temporal stream; making computation and response-time constraints explicit; and treating other agents as part of the environment.[3]

The Alberta Plan’s twelve steps can be organized as a dependency-ordered computational chain. An agent first constructs state, then learns GVFs and continuing control, subsequently constructs subproblems, skills, and transition models from state features, and continually adjusts all of these learning processes through multi-timescale meta-learning. Triadic and dyadic interaction, approximation in a big world, and continual interaction in embodied and large language worlds can all be situated within this chain.

Continual reinforcement learning (CRL) is not simply the sequential training of multiple tasks, nor is it an existing training run made longer. It changes the object of study: from solving a problem whose states, actions, rewards, and boundary conditions have already been specified to understanding a capacity-limited, continually changing actor that receives only one stream of experience.

Over the past decade, deep reinforcement learning has made substantive advances in games, continuous control, complex strategic interaction, robotics, and the post-training of foundation models. These achievements provide indispensable algorithmic foundations for continual reinforcement learning. Yet most mainstream work was not conducted under the problem definition or evaluation objective of continual reinforcement learning. State representations, action sets, rewards, episode boundaries, resets, training phases, and evaluation distributions were typically specified by designers. Modern deep reinforcement learning has primarily addressed **high-dimensional prediction and control within a given problem**. Continual reinforcement learning must additionally explain **how states, knowledge, internal learning objectives, temporal abstractions, and learning mechanisms form from experience and remain open to revision**.

The discussion begins by distinguishing triadic, designer-mediated interaction from dyadic, direct interaction. It then turns to the Big World Hypothesis, the theoretical abstractions of classical reinforcement learning, and the dependency structure connecting agent-state construction, GVF learning, continuing control, transition models, autonomous objective construction, and multi-timescale meta-learning.

1 From Triadic, Designer-Mediated Interaction to Dyadic, Direct Interaction

Traditional reinforcement-learning experiments usually employ **triadic, designer-mediated interaction**: the big world, a designer-constructed environment, and an agent jointly constitute the research system. Continual reinforcement learning ultimately studies **dyadic, direct interaction**: the agent acts directly in the big world and bears the long-run consequences. This distinction follows Sutton’s RLSS 2026 discussion of the triad, the task-giver, and the dyad.[4] The central question is whether states, rewards, task boundaries, and abstractions are specified in advance by a designer or progressively formed by the agent through experience in the big world.

1.1 Triadic, Designer-Mediated Interaction

The conventional experimental structure is closer to

$$W \xrightarrow{D: \text{selection and task construction}} \mathcal{E}_D \longleftrightarrow \text{Ag}.$$

The three objects are the big world W , the designer-constructed experimental environment $\mathcal{E}_D$, and the agent Ag . D denotes the design process, not a fourth learning entity. The designer selects state variables, observation channels, action sets, rewards, termination conditions, reset distributions, and task-switching rules from the world, then packages them into a reproducible environment.

The agent does learn autonomously through trial and error inside $\mathcal{E}_D$, but the interface it receives already contains substantial designer knowledge. Atari frames and action sets, legal Go moves and win conditions, and MuJoCo states and simulation time steps are not discovered by the algorithm at runtime.

This triadic structure is indispensable to experimental science. Without controllable environments, it would be difficult to isolate variables, construct counterexamples, prove theorems, or reproduce experiments. What must remain explicit is the source of capability. High performance in a designer-constructed environment demonstrates that a system can use given states, actions, and rewards to solve a problem. It does not, by itself, demonstrate that the same system can form equally effective states, objectives, and abstractions in a big world.

1.2 Dyadic, Direct Interaction

Research on continual intelligence ultimately concerns a structure closer to

$$W \leftrightarrow \text{Ag}.$$

The agent continually exchanges observations, actions, and consequences with the big world. At runtime there is no task-giver repeatedly announcing, “You are now entering task 7,” “This variable is the state,” or “Optimize this auxiliary loss.” State features, predictive questions, subproblems, skills, and models must increasingly be formed from first-person experience and tested by their contribution to long-run control.

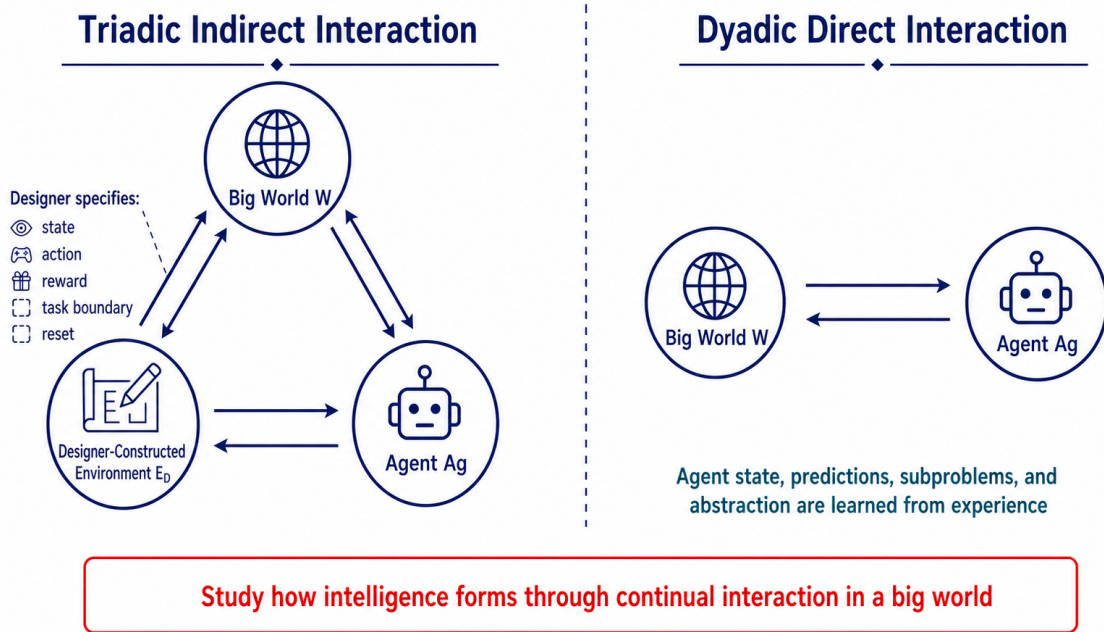


Figure 1 | In the triadic structure on the left, a designer constructs an experimental environment E_D from the big world and specifies states, actions, rewards, task boundaries, and resets. In the dyadic structure on the right, the agent directly bears the consequences of action in the big world; no task-giver continually partitions the runtime into tasks. Redrawn from Sutton’s classroom discussion of the triad, task-giver, and dyad.

Dyadic interaction does not imply that humans disappear from the system. Sensors, actuators, domain-general learning rules, computational resources, safety boundaries, and evaluation institutions remain human-designed. The boundary Sutton emphasizes is closer to *The Bitter Lesson*: general computational and learning mechanisms may be supplied, but features, rules, subgoals, and behavioral answers specific to a world should, as far as possible, be discovered by the agent from experience.[5]

The move from triadic to dyadic interaction therefore does not demand the immediate abandonment of benchmarks. A more productive path is to use triadic systems to analyze mecha-

nisms, test transfer while progressively removing task-specific interfaces, and ultimately evaluate in dyadic interaction whether an agent can continually form useful knowledge from real consequences.

2 The Big World Hypothesis and the Necessity of Continual Learning

Once the object of study moves from a designer-constructed environment back to the big world, an asymmetry appears that cannot be eliminated merely by enlarging a single training run: the action-relevant distinctions in the world far exceed the distinctions a single agent can perceive, remember, represent, and compute at any one time. Javed and Sutton formulate this as the **Big World Hypothesis**.^[6] It is the starting point for the continual-RL problem formulation.

This is not a theorem about every possible problem. Small control systems can be modeled completely, and finite MDPs can admit exact solutions. The Big World Hypothesis selects a different class of research objects. Even as computation grows, the worlds an agent enters, the situations it encounters, the combinatorial actions it can take, and the histories it may need to distinguish also grow. The complexity gap between world and agent does not permanently disappear.

The Big World Perspective

- The world is **vastly more complex** than any AI agent's mind
 - After all, the world contains many people, and what is going on in many of their minds matters for the agent
- It follows that the agent **can never accurately know even the state** of its world
- And simply **can not represent** the optimal policy or any value function
- Much less **learn** them



Figure 2a | The Big World Perspective: even when the world is finite, an agent cannot observe its entire state or represent a globally optimal policy, and must learn approximate representations of state, action, feedback, and update mechanisms. Source: Richard S. Sutton, RLSS Lecture 01, slide 8, “The Big World Perspective.”

Accepting this hypothesis changes four premises of the research problem.

First, the environmental state is ordinarily not fully available. The agent must construct internal state from its observation history rather than identify the current observation with a theoretical Markov state.

Second, a finite agent generally cannot represent a globally optimal policy in full. Optimality remains an exact yardstick in small problems and a valuable reference in large ones; practical research must additionally explain how finite capacity allocates approximation error.

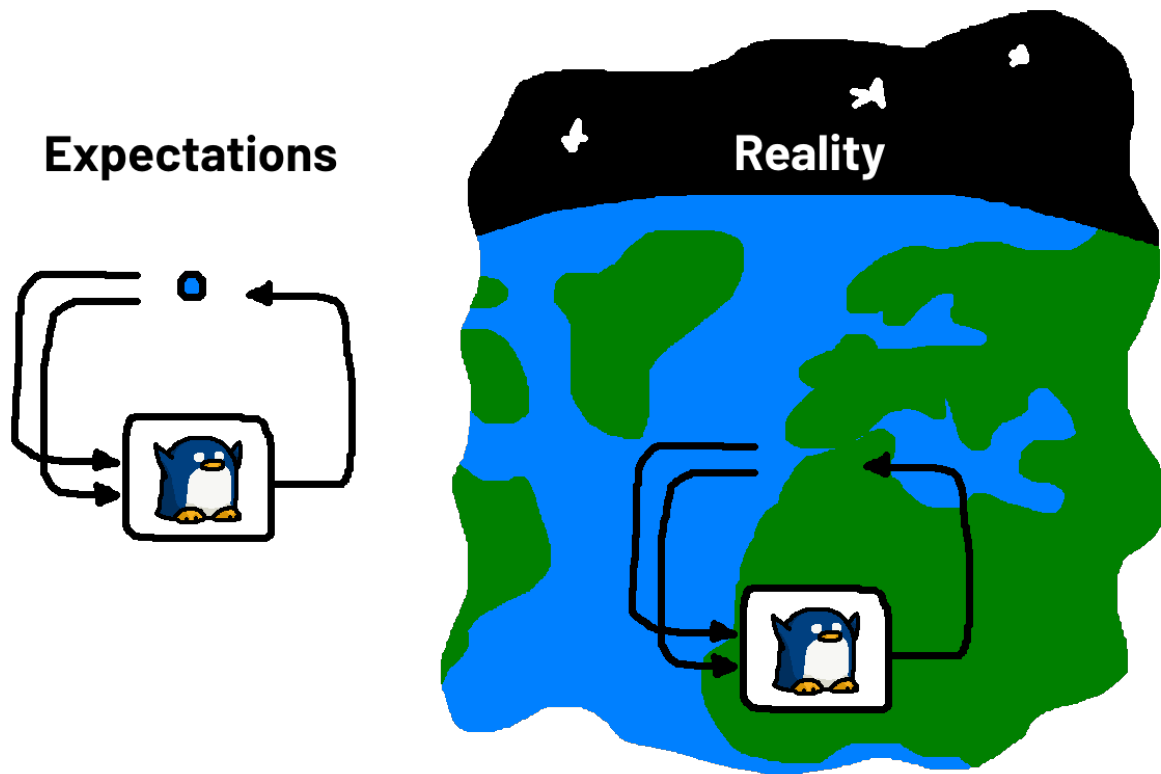


Figure 2b | The gap between a finite loop in expectation and the scale of the real big world. The original penguin agent places the same finite actor in an artificial small world and in an inexhaustible reality. Original figure from Kris De Asis, *Design for Learning* [7]. [7]

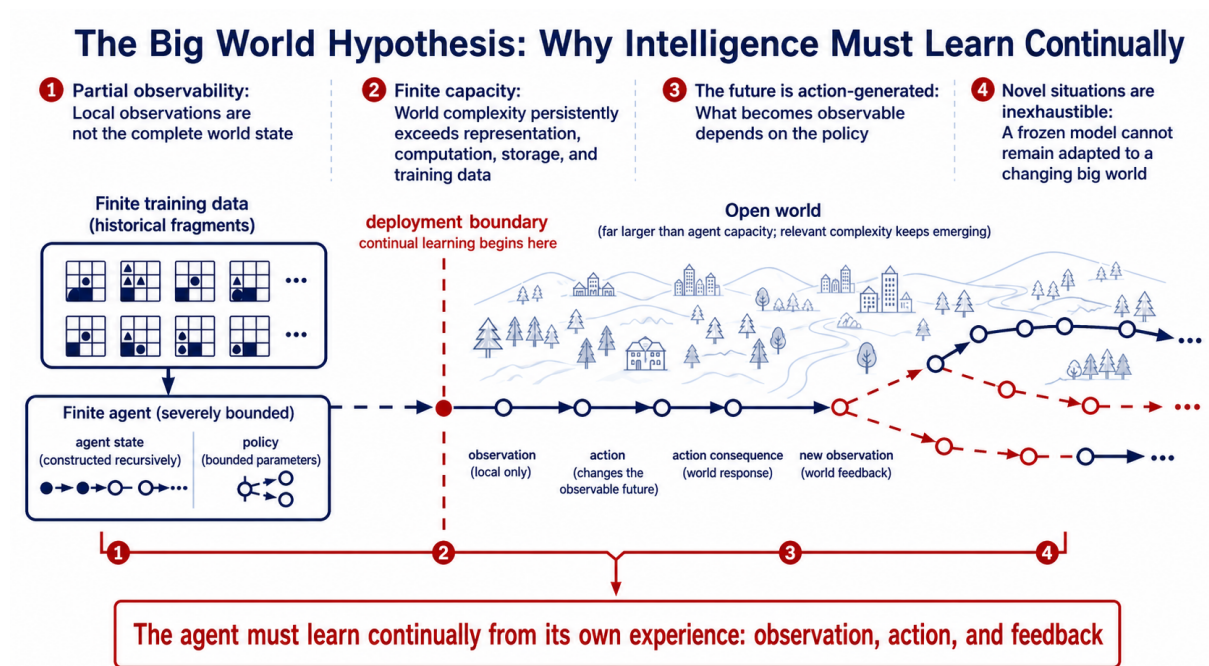


Figure 3 | A finite dataset and a finite agent can cover only a small part of the world. An open world continually produces new states, tasks, interactions, and unenumerable combinations. Intelligence can be revised only through an indefinitely extending stream of observation, action, and feedback.

Third, effective interfaces—including state, action, feedback, and update timescales—must be calibrated through experience. Designer-supplied sensors and actuators are physical interfaces. Which distinctions deserve representation and which consequences deserve prediction remain learning problems.

Fourth, learning cannot treat one-time convergence as its endpoint. The agent must detect when an approximation ceases to work and adjust features, predictions, policies, models, and learning rates.

To **embrace approximation** is not to tolerate arbitrary error or lower theoretical standards. It is to acknowledge that agent states, value functions, policies, and models are capacity-limited working approximations, and to require that subsequent experience can continually test, diagnose, and improve them.[8] Each approximate object should therefore specify its use, operational distribution, update clock, and failure test, rather than report only an average error after a single training run.

2.1 What It Means to Learn from Experience

Static data, expert demonstrations, simulated trajectories, and generated data can all provide strong priors. But the experience of an acting system contains at least an action-conditioned causal chain:

internal state at the time
 → selected action
 → actual response of the world
 → updated tendency to act.

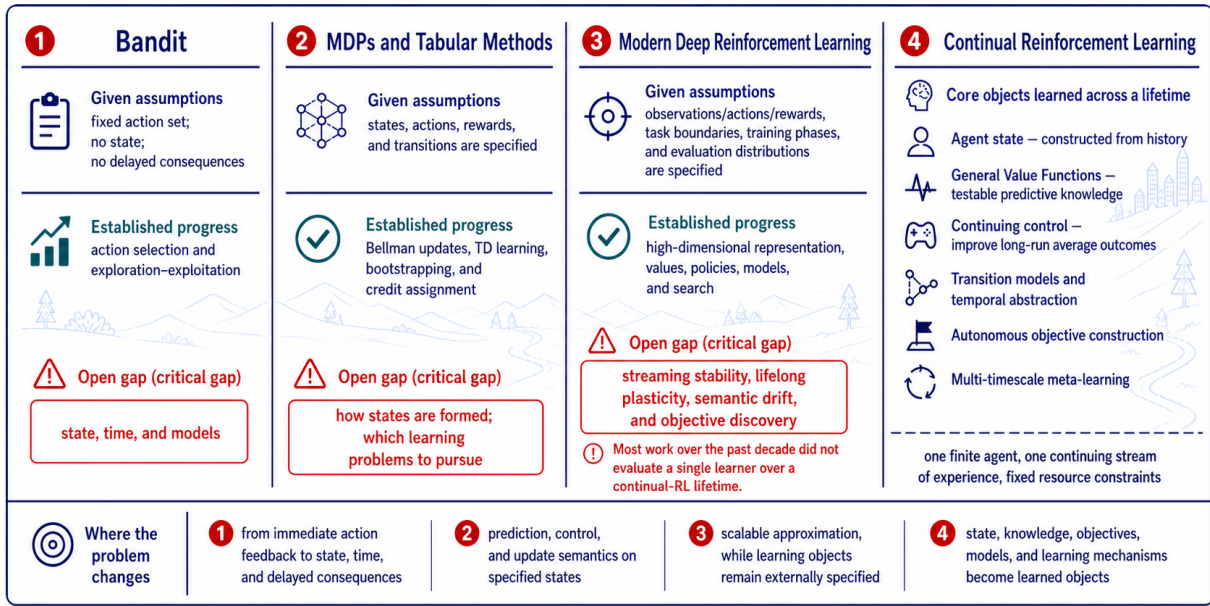
An update constitutes an experiential gain in the continual-learning sense only if it changes later prediction, control, or learning efficiency and is again subjected to real consequences. The **Era of Experience** described by Silver and Sutton concerns long-duration, action-conditioned streams of experience with real consequences—not merely larger static datasets.[9]

Learning from experience does not exclude pretraining, world models, or human knowledge. It requires these priors to remain open to testing by deployment interaction: What did the model predict? Did the action produce the expected effect? Have the conditions under which old knowledge applies changed? After operating longer, does the system make fewer errors of the same kind, or has it merely accumulated more records?

Continual learning is therefore not an optional deployment phase appended after “the large model already knows a great deal.” As long as the world exceeds the agent, actions change future data, and approximations fail over time, learning must remain inside the closed loop.

3 Theoretical Abstractions of Classical Reinforcement Learning and the Reformulation Introduced by Continual RL

Reinforcement-learning textbooks commonly proceed from bandits to MDPs, tabular methods, and deep reinforcement learning. This sequence is not merely chronological. It first uses minimal models to define the learning problem, then studies solution methods after the problem definition has been fixed.



Continual RL changes the problem formulation and learning objective; it is not an added module.

Figure 4 | Bandits and MDPs primarily define problems: what information the agent receives, which actions it may take, how rewards arise, and over what timescale consequences are compared. Tabular methods and deep reinforcement learning primarily supply solution methods: how to estimate value, assign credit, improve a policy, and approximate functions. Continual RL inherits these problem models and solution methods but further changes the problem definition by making agent state, GVF questions, subtasks, transition models, temporal abstractions, and learning mechanisms themselves objects learned from continual experience.

3.1 Define the Problem Before Discussing Solution Methods

At the opening of their textbook, Sutton and Barto distinguish three meanings of reinforcement learning: a class of problems, a set of solution methods suited to those problems, and a field that studies the relationship between the two. The same name may refer to all three, but theoretical analysis must keep them distinct. Otherwise, structure supplied by an algorithm can be mistaken for information supplied by the problem, and the ability to solve a specified task can be mistaken for the ability to define tasks and form knowledge.[10]

A problem definition must answer at least the following: What can the agent observe at each step? Which actions can it take? What consequences does the world return? Which quantities define the objective? How do past actions affect future experience? A solution method instead asks how unknown quantities should be estimated, how exploration should proceed, how temporal credit should be assigned, how a policy should be improved, and how these objects can be approximated under finite computation.

By this criterion, bandits and MDPs are first problem models. Sample averages, UCB, TD, Q-learning, policy gradients, deep networks, and search are solution methods. A problem may be held fixed while algorithms change, and an algorithm may transfer across problem classes. Continual reinforcement learning first changes the former layer: it must specify the problem faced by a lifelong actor before determining which parts can be addressed by tabular updates, deep networks, replay, planning, or meta-learning.

3.2 Bandits: Action Selection, Local Feedback, and Exploration

The basic K -armed bandit specifies a fixed action set $\mathcal{A} = \{1, \dots, K\}$. At time t , the agent selects A_t , observes only the reward R_{t+1} produced by that action, and estimates

$$q_*(a) = \mathbb{E}[R_{t+1} \mid A_t = a].$$

The problem deliberately removes state transitions and delayed consequences. The current action does not alter the situation presented on the next round; it produces only an immediate reward and new information about that action. Because rewards for unselected actions are not observed, the agent receives action-conditioned partial feedback, not a supervised-learning sample with all labels. Exploration is therefore not a training trick but part of the problem structure: selecting the currently estimated best action can yield immediate return, whereas selecting an uncertain action may improve future decisions.

Sample-average and constant-step-size updates determine how action values are estimated. ϵ -greedy selection, optimistic initialization, UCB, and Thompson sampling determine how uncertainty affects action selection. Regret compares the cumulative consequences of the actual action sequence with those of the best fixed action. These are alternative solution methods or evaluation criteria for the same bandit problem; they are not extra information supplied by the bandit environment.

Bandits establish two foundations for the chapters that follow. First, the data distribution is generated by action, and an agent can learn only from the regions it actually explores. Second, online learning must jointly affect present return and future information. Once actions also change the next situation and rewards have delayed effects, independent bandit rounds are no longer adequate; the problem must be extended to an MDP. Continual reinforcement learning asks a further question: can the meanings of actions, the relevant situations, the quantities to estimate, and the aims of exploration themselves change over a lifetime of experience?

3.3 MDPs: State, Time, and Long-Run Consequences

Under the classical discounted-return formulation, a Markov decision process is commonly written as

$$\mathcal{M}_\gamma = (\mathcal{S}, \mathcal{A}, P, R, \gamma),$$

where $\mathcal{S}$ is the environmental state space, $\mathcal{A}$ the action space, P the action-conditioned state-transition process, R the reward process, and $\gamma \in [0, 1)$ the discount factor. It makes the contribution of reward k steps in the future decay as γ^k : $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$. More generally, $(\mathcal{S}, \mathcal{A}, P, R)$ specifies the controlled Markov process, while discounted return or average reward specifies how that process is evaluated across time.

An MDP can describe either an episodic task or a continuing task without a natural endpoint. In a continuing task, $\gamma < 1$ keeps the discounted sum of an infinite reward stream finite, but also introduces an effective timescale determined by γ and progressively downweights distant consequences. Simply setting $\gamma = 1$ ordinarily makes an undiscounted infinite return non-finite, so long-run continuing control often uses average reward as a different control criterion. The discount factor is not itself a termination probability, and the ability of an MDP to describe a continuing process does not depend on the presence of a natural endpoint. The Markov property gives a common language to Bellman equations, dynamic programming, Monte Carlo methods, TD learning, and control.[10]

Here $S_t \in \mathcal{S}$ is an **environmental state variable in the problem model**. It satisfies the Markov condition: given the current state and action, the conditional distribution of the next state and reward does not depend on earlier history,

$$\Pr(S_{t+1}, R_{t+1} \mid S_0, A_0, \dots, S_t, A_t) = \Pr(S_{t+1}, R_{t+1} \mid S_t, A_t).$$

Thus S_t must retain every distinction in the full history that affects future prediction and control. In a standard MDP problem, this state variable and its semantics belong to the problem definition. The learning algorithm uses it; it is not responsible for deciding what should count as state.

This is not the same object as the agent state X_t introduced later:

- **Environmental state** S_t is an analytical variable defined by the researcher in the environment model and assumed to satisfy the Markov property in an MDP.
- **Observation** O_t is the information actually made available to the agent at time t ; it may be a partial, noisy, or selectively constructed mapping of S_t .
- **Agent state** X_t is an internal variable recursively constructed by a finite agent from the observation–action–reward history available to it. It is ordinarily only an approximate state, and both its features and update rule may themselves continue to learn.

In a POMDP, S_t may be an unobserved environmental state while the agent receives only O_t and maintains a belief state or another history representation. The same distinction still applies: the POMDP’s latent state and generative model belong to the external problem description, whereas the X_t actually computed and used for action belongs to the learner. Their numerical values may coincide in a fully observable setting in which the agent directly adopts the environmental state, but their conceptual roles remain distinct.

The theoretical value of the MDP is that, once states, actions, transitions, and the control criterion are given, prediction, control, and temporal credit assignment can be studied precisely. POMDPs, semi-MDPs, non-stationary MDPs, and history-state models can respectively address partial observability, temporal abstraction, environmental change, and history dependence. Continual reinforcement learning does not reject these models. The distinction concerns the object of study and the learning objective, not whether an MDP can describe a continuing process.

3.4 Tabular Methods: Making the Update Semantics of Learning and Control Explicit on Given States

Tabular methods assume finite, discrete, semantically specified state and action sets $\mathcal{S}$ and $\mathcal{A}$, and store a separate estimate for each state or state–action pair. For example, one transition in Q-learning produces the update

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right].$$

This compact equation exposes the principal choices in the solution method. R_{t+1} is the consequence just returned by the environment; the estimate at the next state supplies a bootstrapped target; α determines how strongly new experience changes the old estimate; and $\max_a$ specifies a target policy that may differ from the behavior policy. Replacing the maximizing action with the next action actually taken gives the on-policy semantics of SARSA. Waiting for a complete return gives a Monte Carlo method. Multi-step returns and eligibility traces alter how temporal credit propagates among neighboring events.

Tabular representations do not share parameters across states, so an update at one state does not directly rewrite the numerical estimate at another. This isolation allows researchers to determine whether error arises from sampling, exploration, bootstrapping, off-policy learning, step size, or credit horizon, and supports clear convergence analyses under corresponding conditions. The value of tabular methods is therefore not limited to small problems; they provide a semantic reference for reinforcement-learning updates.

Their limitations are equally clear. State identifiers must be supplied by the problem formulation, unvisited states share no statistical structure, and storage and sampling demands grow rapidly with the state space. Deep reinforcement learning introduces shared parameters to generalize in high dimensions, but one update then affects many states at once, introducing representation drift, interference, and off-policy instability. Continual reinforcement learning asks an earlier question still: if S_t is not supplied to the agent at all, how should the agent state X_t used as the index be formed from history and revised through experience?

3.5 Deep Reinforcement Learning: High-Dimensional Function Approximation on a Given Interface

Function approximation shares parameters across states; deep networks extend this sharing to pixels, language, continuous actions, and large-scale search. DQN demonstrated value-based control from pixels. AlphaGo Zero combined self-play, search, and deep networks. MuZero learned latent dynamics for planning without requiring reconstruction of every observation. Multi-Agent Transformer (MAT) reframed joint policy search in cooperative multi-agent systems as sequential decision making and demonstrated the power of modern sequence models for high-dimensional multi-agent control on specified benchmarks including StarCraft II and Multi-Agent MuJoCo. [Mnih et al. [11]][12][Schrittwieser et al. [13]][14]

These works substantially advanced high-dimensional representation, nonlinear value estimation, complex policy optimization, and the integration of learned models with search. Their typical object of study nevertheless remains a high-dimensional control problem whose observations, actions, rewards, agent roles, episode boundaries, and evaluation distribution have been specified. They can serve as control solvers inside continual reinforcement learning, but they do not automatically constitute mechanisms for runtime state formation, objective construction, structural pruning, or lifelong plasticity.

Experience replay, target networks, parallel environments, normalization, restarts, curricula, and periodic offline training play essential stabilizing roles. But these mechanisms do not separately solve online knowledge selection and deletion under fixed resources, nonlinear off-policy control, continual agent-state construction, semantic drift in shared representations, lifelong plasticity, or the discovery of learning objectives.

3.6 Why Continual Reinforcement Learning Is a Different Problem Formulation

The difference between classical reinforcement learning and continual reinforcement learning is not that the same MDP has a larger state space, a longer runtime, or a deeper neural network. The two formulations take different objects as fundamental and require algorithms to provide different kinds of answers.

Classical reinforcement learning begins with a decision problem: the meanings of states and actions, the reward, transition process, temporal organization, and evaluation criterion are specified by the problem formulation. The algorithm must learn a value function, policy, or model within that problem. Even with online learning, continuing MDPs, POMDPs, non-stationary MDPs, or semi-MDPs, the research question can remain: “How can this already formalized problem be solved more effectively?”

A Fundamental Difference in Problem Formulation: Modern Deep RL Does Not Solve Continual RL in a Big World

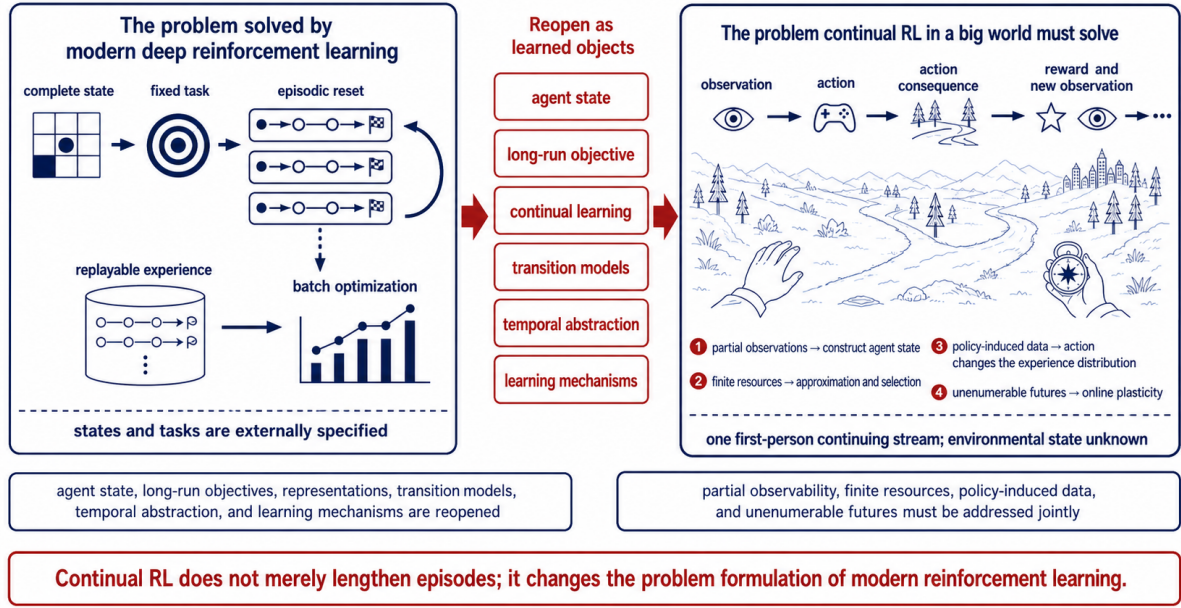


Figure 5 | The fundamental difference between the two research problems. Modern deep reinforcement learning primarily studies local control problems in which states and tasks are externally given. Continual reinforcement learning makes agent state, the long-run control objective, continual updating, transition models, temporal abstraction, and learning mechanisms part of the research object.

The primitive object in continual reinforcement learning is instead a first-person interaction history and a learner that continually changes along that history:

$$H_t = (O_0, A_0, R_1, O_1, \dots, A_{t-1}, R_t, O_t),$$

where O_t , A_t , and R_t are the observation, action, and scalar reward. This follows the Alberta Plan's treatment of experience as three temporally aligned sequences of observations, actions, and rewards. Throughout this essay, **Ag** denotes the agent, $A_t \in \mathcal{A}$ a primitive action, $O_t \in \mathcal{O}$ an observation, and $\omega \in \Omega$ an option extending across multiple time steps. The environment is not obliged to supply a sufficient state, a fixed task identifier, a reproducible initial-state distribution, or a natural termination time. Safety constraints, computational resources, and other evaluation variables must be specified separately in the experiment rather than casually folded into the reward symbol.

The two problem formulations can be summarized as follows:

Dimension	Classical RL within a specified problem	Continual reinforcement learning
Primitive object	A formalized decision process and samples from it	A continuing stream of experience in a big world and a learner that changes
State	S_t is defined by the problem model and satisfies its assumptions	X_t is constructed from available history by the agent and continually revised
Learning problems	States, actions, rewards, prediction quantities, and task boundaries are usually specified	Predictive questions, subproblems, skills, models, and temporal abstractions must also form from experience

Dimension	Classical RL within a specified problem	Continual reinforcement learning
Organization of time	Episodes, resets, training phases, and evaluation distributions may be experimentally specified	The same agent undergoes uninterrupted interaction; past updates continue to affect future learning
Object to be solved	A value function, policy, or model under a given criterion	The process by which a complete learner forms and revises state, knowledge, control, models, and learning mechanisms
Principal failure modes	Estimation error, insufficient exploration, optimization error, or function-approximation error	In addition: state failure, stale knowledge, structural growth, loss of plasticity, and mismatch among learning objectives

Replacing an MDP with a continuing MDP changes whether the environmental process has a natural endpoint. A POMDP makes environmental state partially observable. A non-stationary MDP permits transition or reward dynamics to change over time. A semi-MDP permits decisions of unequal duration. All of these extensions are important, but none automatically makes state construction, generation of predictive questions, discovery of subproblems, structural pruning, and multi-timescale meta-learning objects that the algorithm must solve. Continual reinforcement learning differs precisely because objects ordinarily fixed by the problem formulation or the surrounding system are brought into the continual formation of one learner.

The learning object of continual reinforcement learning is an agent that changes through experience. Its long-run objective is not to compute the optimal policy of one fixed MDP, but, through continual interaction, to

- construct from first-person history an agent state useful for action;
- form general value predictions testable by future experience;
- learn continuing-control policies that improve long-run average outcomes;
- learn transition models for actions and skills, and autonomously formulate and solve reusable subproblems; and
- regulate learning speed, credit, structure, resources, and plasticity across multiple timescales.

MDPs, POMDPs, semi-MDPs, non-stationary MDPs, and history-state models can describe local environmental structure, support theoretical analysis, and define experimental problems. They cannot substitute for a definition of the complete learner in continual reinforcement learning. Continual RL centers on the process through which a learner forms state, predictions, control, models, objectives, and learning mechanisms in continuing interaction. A method capable of solving each specified MDP does not thereby possess the ability to form these learning objects autonomously in a big world.

4 Overall Computational Structure of Continual Reinforcement Learning in a Big World

Supervised, unsupervised or self-supervised, and reinforcement learning are commonly distinguished by their data and feedback signals. For a continuing actor, it is also necessary to distinguish five interdependent learning processes by the functions the system must perform:

1. **Agent-state construction:** form internal state from first-person history;

2. **General Value Function learning:** form testable and reusable general value predictions;
3. **Continuing-control learning:** improve long-run outcomes through action while changing the distribution of future experience;
4. **Transition models, multi-step skills, and high-level planning:** autonomously formulate subproblems, construct options (denoted ω , temporally extended actions that follow internal policies for multiple steps), and compare long-run consequences across timescales; and
5. **Multi-timescale meta-learning:** regulate the learning speed, credit assignment, structure, resources, and plasticity of the first four processes.

In this decomposition, “representation learning” is made concrete as agent-state construction. The aim is not a generic embedding, but the retention of historical distinctions useful for prediction, control, modeling, and planning. “Learning-objective construction” spans GVFs and temporal abstraction: the agent must generate candidate GVF questions and subproblems, test whether downstream decisions actually use them, retain effective objects under finite capacity, and remove ineffective ones. A learning objective is therefore not a designer-specified list of auxiliary losses; it is part of the continual learning process itself.

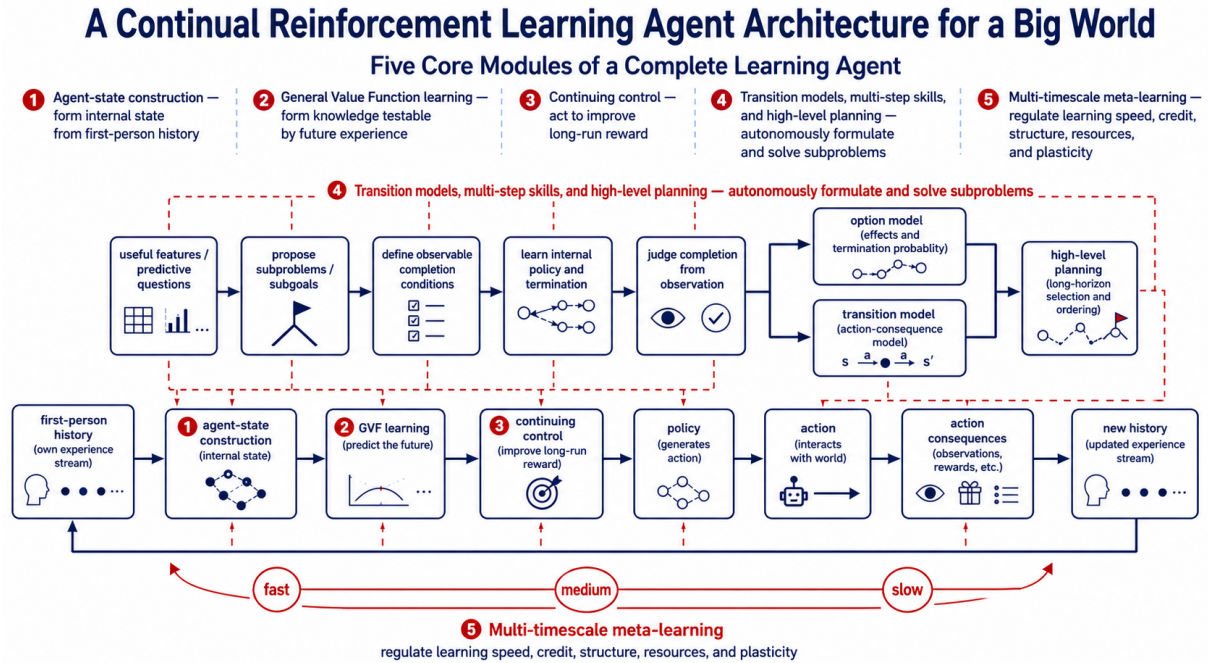


Figure 6 | The five learning processes form a dependency-ordered closed loop. Agent state supplies a common coordinate system for GVFs, control, and models; GVF knowledge supports action, planning, and skills; control changes state visitation; models and temporal abstractions expand the reuse of experience; and meta-learning regulates the long-run evolution of all modules.

This structure is connected to Sutton’s common model of the intelligent decision maker, which unifies perception and representation, prediction, action and value, and transition models, while extending it by placing agent-state construction, autonomous learning objectives, and multi-timescale meta-learning within the same continuing process.[15]

A minimal computational chain is

$$H_t \xrightarrow{U_t} X_t \xrightarrow{\text{prediction, control, model, planning}} A_t \xrightarrow{W} (R_{t+1}, O_{t+1}) \xrightarrow{\text{learning and meta-learning}} \Xi_{t+1}.$$

X_t is the agent state read by the current action-selection process. Ξ_t is the complete internal learning state that allows the entire learning process to continue, including parameters, memory,

eligibility traces, optimizer state, predictive questions, models, skills, and version information. Identical current policy outputs do not imply that two agents have the same Ξ_t , or that they will learn in the same way when the next experience arrives.

The next five sections consider these five learning processes in order. The sequence cannot be permuted arbitrarily. Without agent state, prediction, control, and models lack a common coordinate system. Without predictive and control uses, state features cannot be tested by experience. Without action-conditioned predictive knowledge, temporal abstractions and planning lack calibrated consequences. Without the first four classes of objects, meta-learning has no basis for deciding what to regulate.

5 Agent-State Construction: Forming Internal State from First-Person History

The complete environmental state of a big world is unavailable. A current observation is usually a lossy perceptual window; a context window, external memory, or state estimator likewise supplies only an approximation to a finite portion of history.

A finite agent must recursively construct its own internal state:

$$X_t = U_{\psi_t}(X_{t-1}, A_{t-1}, R_t, O_t), \quad A_t \sim \pi_{\theta_t}(\cdot | X_t).$$

X_t is the **agent state**, not the Markov state S_t defined by the environment model in Section 3.3. S_t describes which historical distinctions in the environment affect the future. When the complete S_t is unknown, X_t is the finite, computable approximation the agent constructs to those distinctions. If two histories h and h' map to the same x , while an action has different consequences after those histories, then every prediction, policy, or model that depends only on x necessarily contains state-aliasing error.

Increasing the number of network parameters can improve function approximation on the supplied input, but it cannot recover information already discarded by the state update. Adding more text to a context likewise does not guarantee an approximately sufficient state: the window truncates history, retrieval recalls selectively, external memory becomes stale, and workflows often encode task semantics in advance through designer decisions.

Evaluation of state construction must be linked to downstream use. A feature may improve short-horizon prediction while degrading long-run control; it may help value estimation while inducing greater transition-model bias. There is therefore no unique “optimal representation” independent of predictive questions, control objectives, and model uses.

Agent state must also remain revisable. When sensors, tool versions, embodiment, institutions, or policies change, a compression that was once effective may no longer be sufficient. Continual state construction studies precisely this recursive, use-dependent internal coordinate system whose errors can be corrected by later experience.

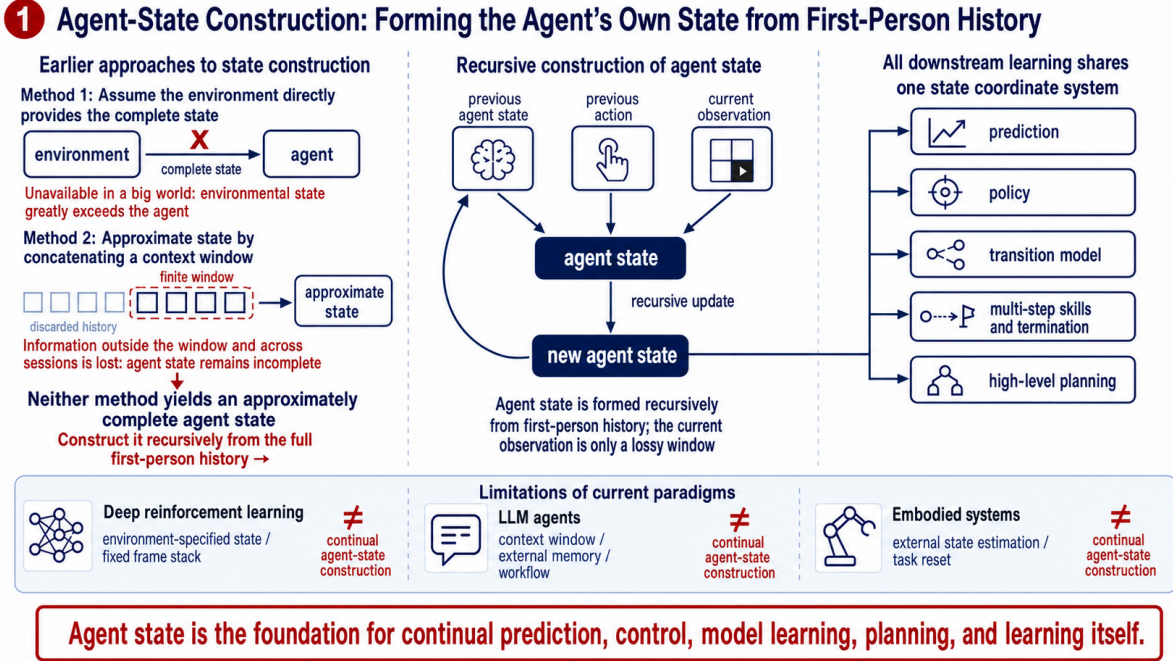


Figure 7 | Agent state is formed recursively from the previous state, previous action, and current observation. Prediction, policy, transition models, multi-step skills, and high-level planning share this state coordinate system and test, through their respective future uses, which historical distinctions should be preserved.

6 General Value Function Learning: Testable and Reusable Predictive Knowledge

A classical value function asks how much cumulative reward will be obtained when a policy is followed. A continual agent needs a broader family of predictive questions, including changes in sensors, the occurrence of events, risk, reachability, duration, termination conditions, and consequences at multiple timescales.

General Value Functions (GVFs) unify such questions as

$$V^{\pi, C, \gamma}(x) = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \left(\prod_{j=1}^k \gamma_{t+j} \right) C_{t+k+1} \middle| X_t = x \right],$$

where π specifies the target policy under which the question is asked, C is the stepwise **cumulant** specifying the signal to be predicted, and γ is the **continuation function** specifying the temporal semantics of continuation and termination. A conventional reward value function is a special case of a GVF. More general GVFs can predict sensor events, risk, reachability, duration, and future consequences under different policies.

An agent might predict the probability of overheating within ten minutes, what it will observe if it continues its current policy, how long it will take to reach an obstacle or goal, how risk changes under another policy, or whether permissions will be invalidated after a rollback. Horde demonstrated an architecture for learning many such predictions in parallel from sensorimotor experience.[16]

This knowledge has four necessary properties:

- **First-person:** it uses only the history and state available to the agent at the time;
- **Policy-conditional:** it makes explicit the behavior under which the future is predicted;

2 General Value Function Learning: Testable, Reusable Predictive Knowledge

Classical value prediction: how much reward will be obtained under a policy => **General Value Functions:** predict sensors, events, risk, reachability, termination, and multiple timescales

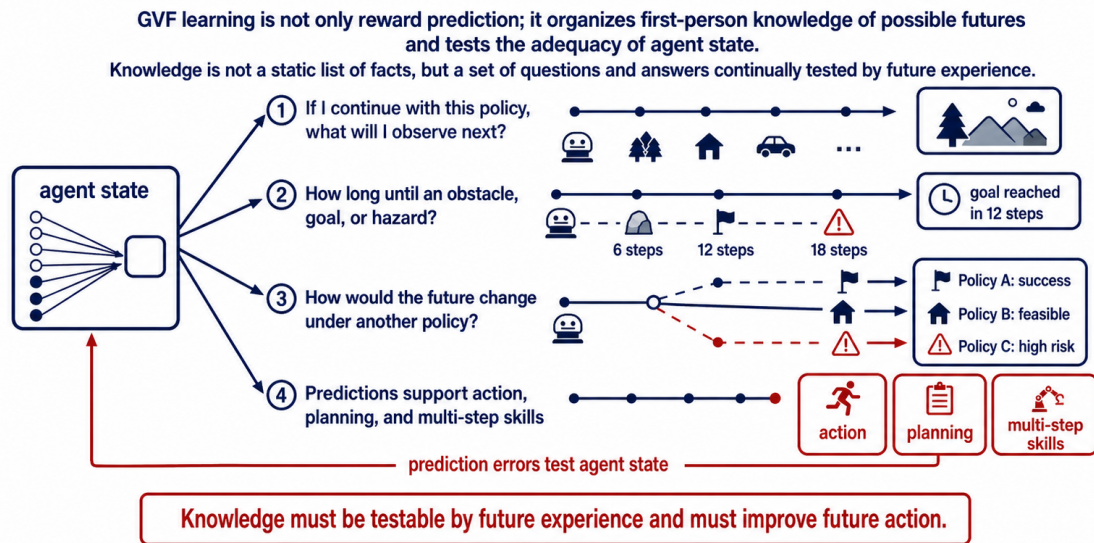


Figure 8 | GVF learning organizes first-person futures as testable questions with explicit target policies, cumulants, and termination semantics. Prediction error both produces knowledge and tests agent state. A general value prediction becomes reusable knowledge only when action, planning, or skill learning actually consumes it and later consequences improve.

- **Empirically testable:** later events can calibrate or falsify the prediction; and
- **Consumed downstream:** state construction, a policy, a planner, or skill learning actually reads the prediction.

A reduction in prediction error establishes improved calibration on the corresponding question, but does not by itself establish usefulness for control. A prediction must be tested through the causal chain “prediction changes a decision → the decision changes action → action produces a consequence.”

7 Continuing-Control Learning: Action Selects Both Outcomes and Future Experience

Control learning selects actions through a policy in order to improve long-run outcomes. The **Reward Is Enough** hypothesis proposes that capabilities such as perception, knowledge, language, and skills can arise as means to maximizing long-run reward rather than as separate final objectives.[1] The hypothesis does not claim that an arbitrary reward design is sufficient, nor that a continual agent needs only one scalar value head.

The environmental outcome signal, long-run control criterion, internal learning problems, and research evaluation variables must remain distinct:

1. At each step, the environment returns a scalar reward R_{t+1} . Costs, constraints, and safety outcomes should enter the reward when the problem definition so specifies, or be reported as separate evaluation variables.
2. The long-run control criterion specifies how complete lifetimes are compared.
3. Internal predictions, subproblems, and skills are learning objects used to solve the long-run control problem.

4. Lifetime return, recovery time, late-stage learning speed, and resource use evaluate the complete learning process.

7.1 From Discounted Return to Long-Run Average Outcomes

Discounted return

$$G_t^\gamma = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}, \quad 0 \leq \gamma < 1$$

applies exponential attenuation to the future, with a commonly used effective horizon of approximately $1/(1 - \gamma)$. For $\gamma = 0.99$, this is about 100 steps; for $\gamma = 0.999$, about 1,000 steps. Discounting is appropriate for specifying finite time preference, random termination, or local predictive questions, but attenuates the value of distant wear, information acquisition, irreversible change, and later plasticity.

Naik and colleagues show that, under standard formulations of continuing control with function approximation, statewise discounted “optimality” need not correspond to a well-defined global optimization objective.[17] For continuing interaction without a natural endpoint, a more natural reference criterion is the long-run average reward rate:

$$\bar{r} = \liminf_{T \rightarrow \infty} \frac{1}{T} \mathbb{E} \left[\sum_{t=0}^{T-1} R_{t+1} \right].$$

Average reward is not by itself a complete objective for continual learning. A complete evaluation must also consider cumulative opportunity loss, recovery time after change, late-stage learning speed, constraint violations, tail risk, computation, storage, and human intervention.

The relationship between discounted return and continual reinforcement learning therefore requires careful qualification. A fixed $\gamma < 1$ can define a local control problem, a predictive horizon, or the termination semantics of an option, but it cannot by itself carry the objective of a complete lifetime. It evaluates future reward on one fixed geometric scale and does not directly evaluate which state representation, knowledge coverage, and plasticity an update preserves for the more distant future. Two agents can obtain the same return over the current discounted horizon yet differ radically in adaptation and recovery when the environment changes later. Continual RL retains the local uses of discounted prediction while evaluating the complete learning process through long-run average outcomes, opportunity loss, adaptation speed, risk, and resource cost.

7.2 A Policy Changes What the Learner Can Learn Next

Even when external regularities remain unchanged, a changing policy π_t changes the state-visitation distribution. After learning a new route, subsequent data increasingly comes from that route. Once the agent stops trying a class of actions, corresponding counterfactual evidence gradually disappears. The behavior policy determines which data actually occurs; the target policy determines which future the agent seeks to predict or improve. A larger network cannot identify a target outside the available coverage. Bootstrapping, off-policy data, and function approximation can additionally form the **deadly triad**.[10]

The learning objective of continuing control therefore has two coupled effects. Actions should improve long-run external outcomes; the experience generated by those actions should also support further improvement in state, predictions, and models. The second effect cannot be optimized independently of the first: “collecting more data” can otherwise incur unacceptable risk, cost, or long-run damage.

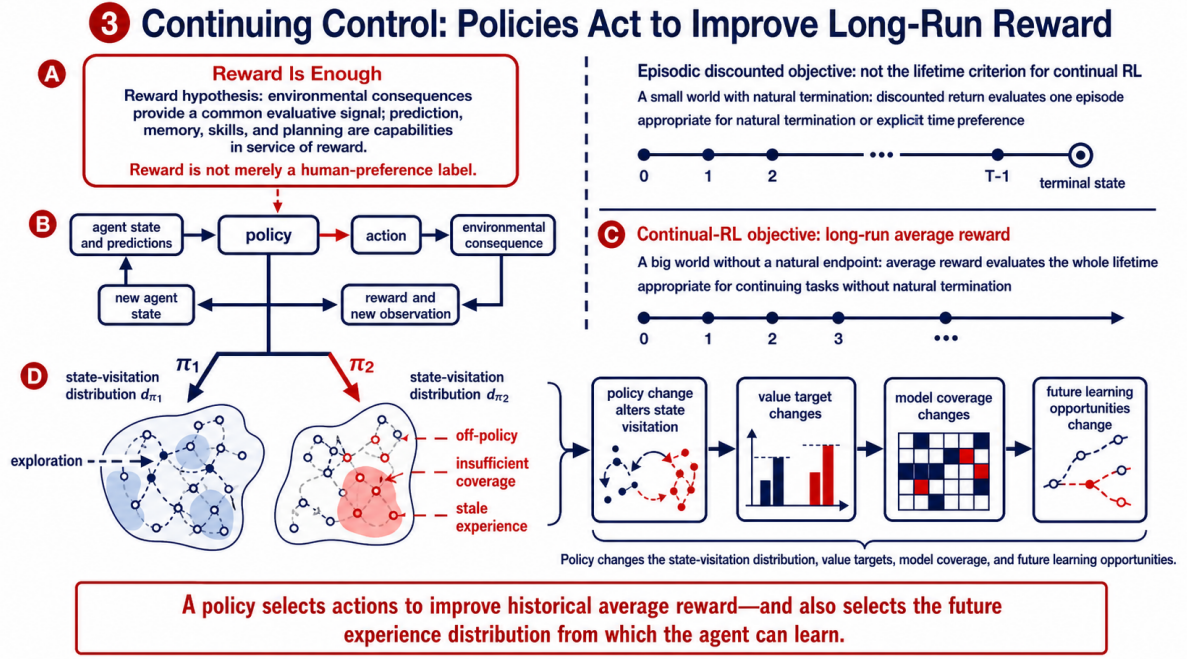


Figure 9 | Continuing control concerns long-run average outcomes when there is no natural endpoint. A policy change simultaneously changes state visitation, value targets, model coverage, and future learning opportunities; control therefore cannot be evaluated only by immediate action return.

8 Transition Models, Temporal Abstraction, and Autonomous Objective Construction

A transition model learns action-conditioned consequences on the agent's internal state:

$$\hat{p}(X_{t+1}, R_{t+1} \mid X_t, A_t).$$

The model is not a complete replica of the external world. It is a collection of action-conditioned predictions that serve control and planning and that later experience can calibrate. For an option ω , the model additionally predicts cumulative reward during execution, the terminal agent state, and duration.

8.1 One-Step Models and the One-Step Trap

If state were sufficient, the model exact, and computation unbounded, repeated application of a one-step model could produce predictions at any horizon. In a big world, every model is approximate. One-step error is repeatedly reused during rollout, search branches grow rapidly with depth, and long-horizon consequences depend on the intermediate policy.

8.2 What Is Constructed Autonomously Are Internal Learning Problems, Not New External Rewards

Autonomous objective construction first requires four objects to remain distinct. External reward R_t defines the ultimate direction of control. A GVF question specifies what to predict under a policy and timescale. A **subtask** specifies a learnable local control problem. Research evaluation variables determine whether the complete agent has improved. Continual reinforce-

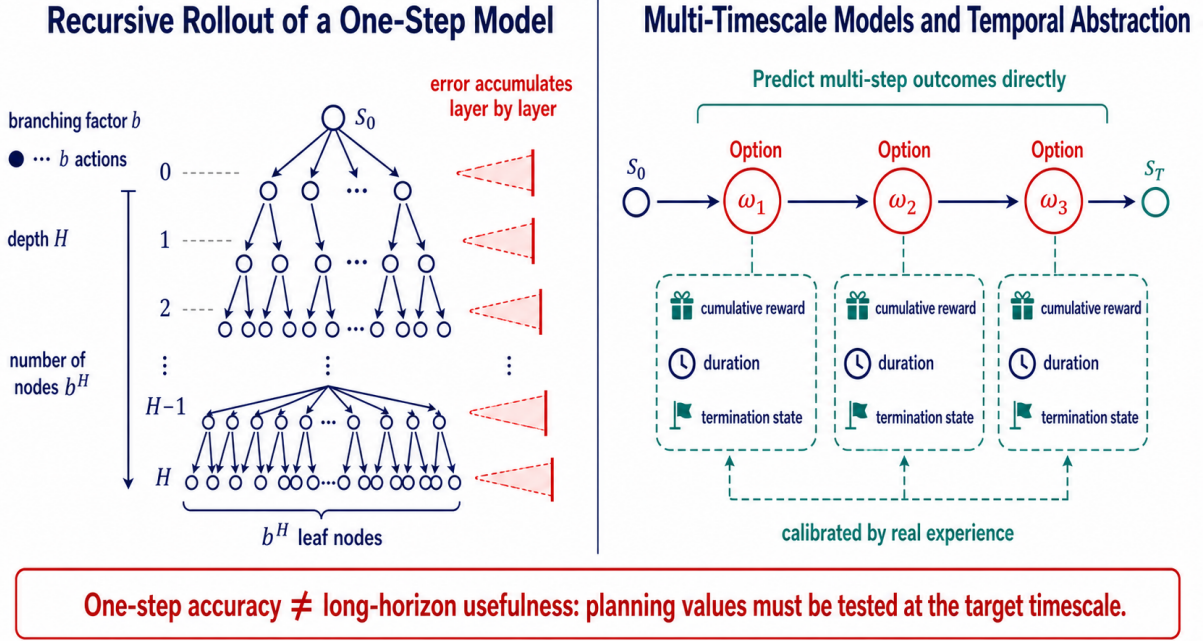


Figure 10 | A one-step model remains necessary, but one-step accuracy cannot substitute for multi-step calibration. When a primitive-action model is rolled out recursively, search size grows with branching factor and depth, while approximation error is repeatedly reused. Multi-timescale models and option models directly predict cumulative reward, duration, and terminal state at the target horizon. Redrawn from Sutton’s discussion of the one-step trap in the RLSS 2026 OaK material.

ment learning must autonomously construct internal predictive questions and subtasks, rather than continually add independent final rewards.

Candidates may arise from state features, learned GVFs, uncertainty, long-run bottlenecks, recurring events, failure-recovery processes, or local structures repeatedly visited by a planner. But being easy to name or frequently encountered supports only candidate generation; it does not establish usefulness. A candidate internal problem must specify at least the predicted or attained quantity, the applicable policy condition, the timescale, an observable termination condition, an initiation region, the required computation and real-interaction cost, and the downstream learning module that will use its answer.

Candidate objectives require six kinds of tests:

1. **Definable from experience:** the target quantity and termination event can be identified from observations, actions, and rewards actually available to the agent.
2. **Currently learnable:** the behavior policy supplies necessary coverage, and the current state and function class can reduce error under finite resources.
3. **Reward-respecting:** the primary reward accrued during execution, time cost, and continuation value after termination are fully included; an arbitrary intrinsic reward must not overwrite the long-run control objective.
4. **A maturity period:** the candidate is first learned through an isolated path so that an unstable estimate does not immediately alter the current policy.
5. **An explicit consumer:** the prediction or skill is actually read by state construction, a critic, actor, transition model, or planner.
6. **Positive long-run net utility:** under the same real-interaction and computation budgets, gains in prediction, control, or planning exceed the costs of exploration, learning, storage, and interference.

Let a candidate feature be $\phi_i(X)$, the long-run average reward rate of the primary control process be $\bar{r}$, the differential value be $\tilde{v}$, and the internal policy of a candidate option ω_i be π_{ω_i} , terminating at a random time T . A reward-respecting feature-attainment objective can be written as

$$J_i(x) = \mathbb{E}_{\pi_{\omega_i}} \left[\sum_{k=t+1}^T (R_k - \bar{r}) + \tilde{v}(X_T) + \kappa_i \phi_i(X_T) \middle| X_t = x \right].$$

This objective jointly accounts for primary reward during execution, the opportunity cost of time relative to the average reward rate, continuation value after rejoining the primary task at the terminal state, and the benefit of attaining the candidate feature. Deviation from the current policy is justified only when the feature benefit compensates for detours, elapsed time, and undesirable termination. This is the fundamental difference between a **reward-respecting subtask** and an arbitrary intrinsic reward.

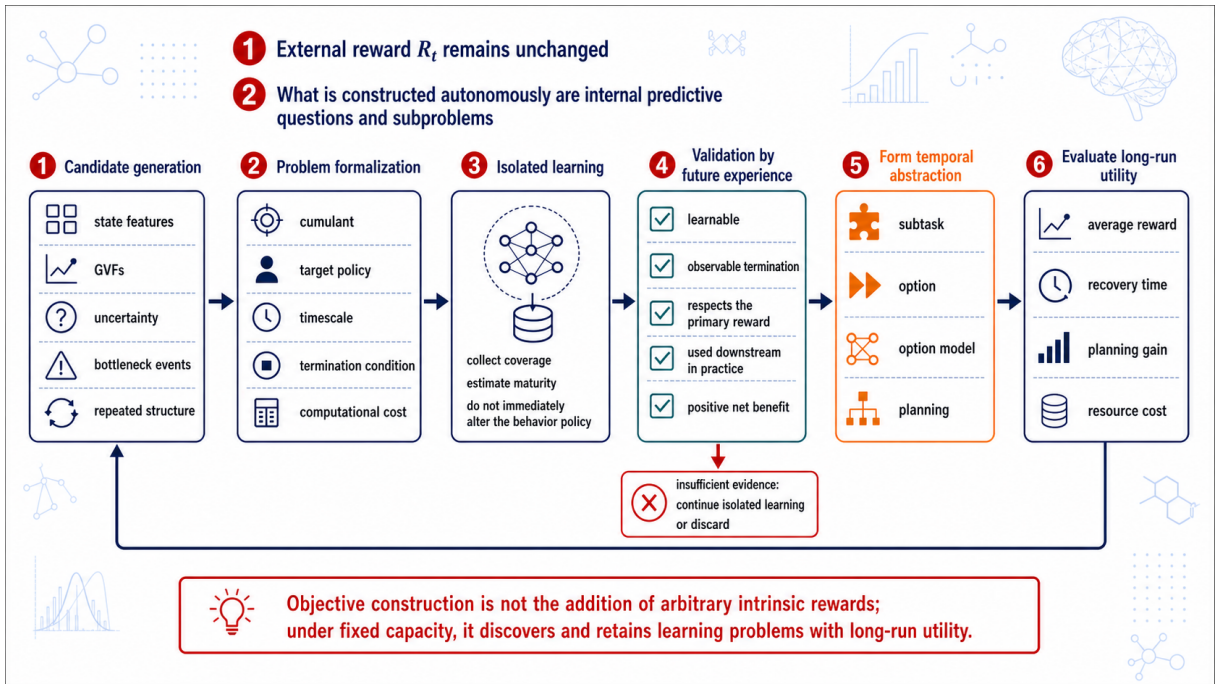


Figure 11 | External reward remains unchanged. Internal predictive questions and subtasks are constructed through candidate generation, problem formalization, isolated learning, validation by future experience, formation of temporal abstractions, and evaluation of long-run utility. Candidates that fail the tests remain isolated or are pruned. Candidates that pass enter options, option models, and planning, and are further evaluated by long-run average reward, recovery time, and realized planning gain. The structure synthesizes the Alberta Plan’s STOMP progression and the Oak feedback process.[3]

Open-ended multi-agent learning has already developed several operational mechanisms for generating learning problems. **Neural Auto-Curricula** parameterizes both which opponents to interact with and how a population should be updated, then learns curriculum-update rules by meta-gradients using exploitability. The curriculum can therefore become a learning object rather than remain entirely hand-specified by researchers.[18]

In non-transitive games, repeatedly pursuing only the current best response can create policy cycles and lose coverage of the existing policy space. Behavioral-diversity methods use occupancy distributions, response differences, or expansion of the gamescape to characterize complementarity among candidate policies, so the generation of a new policy considers both current performance and population coverage.[19] COLE further constructs open-ended learning objectives in cooperative games from compatibility relationships among policies and continually

discovers gaps in the population’s cooperative coverage, improving coordination with unseen partners.[20]

These methods share a common structure: generate a new problem from the current population and learning state, solve it through a relatively independent learning process, and use exploitability, policy-space coverage, or cooperative compatibility to determine whether it should enter the subsequent curriculum. MALib organizes task assignment, sampling, learning, and evaluation as a scalable population-based training process.[21] In continual single-agent learning, opponent or partner selection is replaced by GVFs and reward-respecting subtasks proposed from first-person experience. Population-level exploitability or compatibility is correspondingly replaced by the realized contribution of those internal problems to long-run prediction, control, and planning.

8.3 Options and Temporal Abstraction: From Subproblems to Multi-Step Skills and High-Level Planning

A primitive action specifies what to do at the next time step. An **option** treats a possibly multi-step, closed-loop course of action that responds to intermediate state as one choice available to a higher-level policy. A standard option is

$$\omega = (\mathcal{I}_\omega, \pi_\omega, \beta_\omega) \in \Omega,$$

where $\mathcal{I}_\omega$ is the initiation set specifying the agent states in which the option can be selected; π_ω is the intra-option policy that continually selects primitive actions while the option executes; and $\beta_\omega(x) \in [0, 1]$ is the termination function specifying the probability of ending upon reaching internal state x . After the higher-level policy selects ω at time t , π_ω produces $A_t, A_{t+1}, \dots, A_{t+K-1}$ until termination after a random duration K . Only then does the higher-level policy select another primitive action or option. A primitive action is a degenerate option that lasts one step and then terminates with probability one.[22]

For example, “go to the kitchen and get water” is not a primitive action. It may include navigating around a chair, adjusting speed, opening a door, and approaching a cup, while changing route in response to intermediate observations. Whenever the agent is in a state from which fetching water can begin, the behavior can be represented as an option with an initiation set, an internal navigation-and-manipulation policy, and termination rules such as “water acquired,” “failure,” or “timeout.”

Temporal abstraction means representing, learning, and selecting actions at multiple timescales. A lower-level policy still controls execution at every step, whereas a higher-level policy treats an option ω lasting K steps as one decision unit. The corresponding option model need not roll out every intermediate state. It can directly predict

$$\hat{p}_\omega(X_{t+K}, G_{t:t+K}, K \mid X_t),$$

the agent state at option termination, cumulative reward during execution, and duration, optionally together with risk and uncertainty. “Reach the doorway,” “run the complete test suite,” or “return to the charging station” can then enter planning as abstract actions. The effective depth of a long-horizon decision shrinks from many primitive steps to fewer option choices. Temporal abstraction is not merely trajectory compression. It is the learning of which cross-step behaviors deserve to become reusable decision units, together with continual calibration of their applicability, termination conditions, and models by real consequences.

The formation process has a definite order:

- candidate feature or prediction
- reward-respecting subproblem
- observable completion and termination conditions
- intra-option policy
- option model
- high-level planning.

4 Transition Models, Multi-Step Skills, and High-Level Planning: Autonomously Formulating and Solving Subproblems

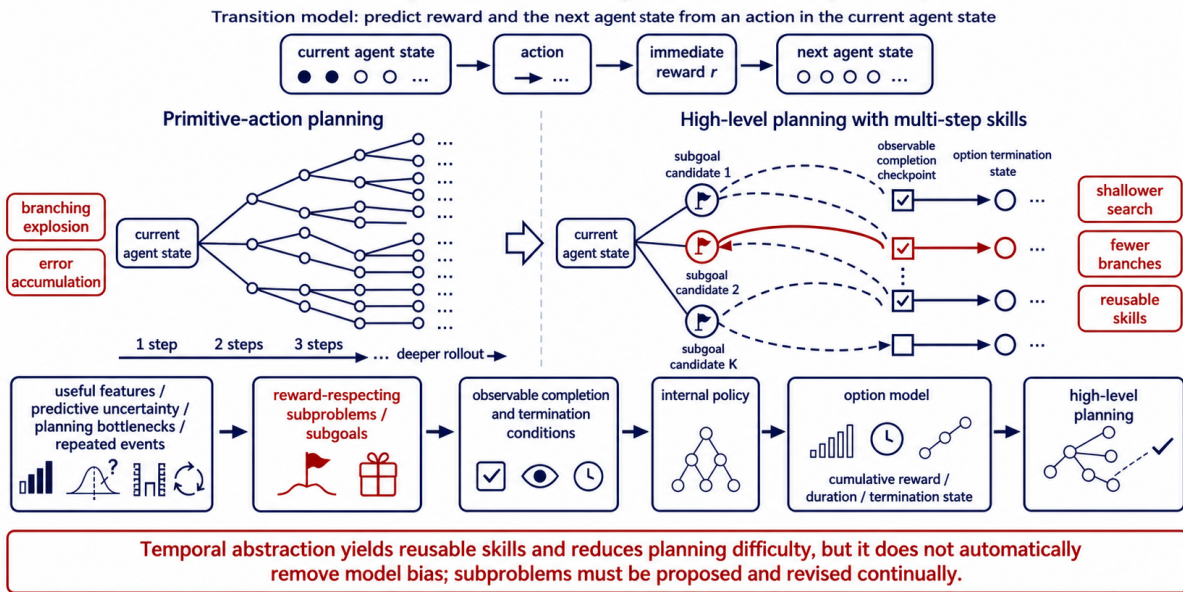


Figure 12 | Temporal abstraction reduces search depth and branching by autonomously formulating subproblems and learning intra-option policies and option models. Option models remain biased approximations; candidate objectives, termination conditions, policies, and models must therefore remain open to testing and revision by real consequences.

The Alberta Plan’s STOMP progression gives a clear dependency chain: form a reward-respecting subtask from a high-utility state feature, solve the subtask to obtain an option, learn an option model, and use the model in planning. OaK adds continual evaluation of utility: if an option model is not used by planning over time, the associated option, subtask, and state feature should be reprioritized or replaced.[3] Objective construction thereby becomes a problem of structural selection under finite capacity, not the unlimited addition of auxiliary tasks. OaK currently supplies a research architecture connecting state features, subproblems, options, option models, and planning. How all of these components should learn jointly during large-scale continual interaction remains an open problem.[4]

9 Multi-Timescale Meta-Learning: Keeping the Complete Agent Able to Learn

Each of the first four learning processes can fail over time. State features become obsolete, predictive questions lose their consumers, policies change data coverage, and transition models become mismatched as skills and the world change. Meta-learning must use long-run experience to regulate the learning processes themselves.

9.1 Core Difficulties When Deep Function Approximation Enters Continual Reinforcement Learning

Deep networks provide high-dimensional nonlinear function approximation. But “can fit a complex function” and “can keep learning from one long stream of experience” are different propositions. When one deep representation jointly supports state, predictions, values, policies, and models, the central difficulties can be grouped into three classes.

First, network capacity cannot replace the construction of agent state and learning problems. If the state update does not retain a historical distinction that determines future consequences, a larger network can still produce only one answer for the same input; parameters cannot recover discarded information. Backpropagation likewise answers only how parameters should change under a given loss. It does not decide which history should be retained, which futures should be predicted, which subproblems should be formed, or which internal objects deserve finite resources. State construction and selection of learning problems must therefore be tested by their later uses in prediction, control, and planning; they cannot be derived from representation dimension or training loss alone.

Second, per-transition deep learning interacts with the moving targets of reinforcement learning. A continual agent receives temporally correlated data produced by its current policy, with a distribution that keeps changing; value targets also depend on estimates that are themselves being updated. When off-policy data, bootstrapping, and function approximation interact, an update on one sample can alter predictive targets at many states simultaneously. Replay, target networks, and batch updates can moderate correlation and rapid target drift, but they also introduce repeated use of data, historical sampling distributions, and lagged parameter copies. Stream-x shows that, when states, network architecture, and learning objectives are given, suitable initialization, normalization, scale control, and update constraints can allow deep prediction and control to cross the batch-size-one **stream barrier**. This establishes the eligibility of a streaming base learner; it does not establish that states, objectives, and temporal abstractions can themselves form continually.[23]

Third, lifelong learning must jointly address retention, plasticity, and consistency of representational semantics. Catastrophic forgetting means that new learning damages old function; loss of plasticity means that the same network becomes progressively less able to learn new regularities that remain learnable later in its lifetime. These must be tested separately by reintroducing old regularities and introducing new regularities late in learning. Long-run deep learning also faces semantic change in representations: after a hidden feature changes meaning, the value weights, policy weights, eligibility traces, optimizer state, normalization statistics, transition models, and planning caches that depend on it may continue to operate under the old meaning. Continual Backprop and related methods maintain network diversity by replacing low-utility units, supplying an important mechanism for plasticity. They do not by themselves guarantee retention of old knowledge or synchronized updates to every downstream learning state.[Dohare et al. [24]][25] Whether a representational change is beneficial must ultimately be judged by named predictions, control outcomes, model calibration, and planning gains under a fixed budget—not only by unit activity or a local loss.

These three classes of problems also require system-engineering improvements and learning-mechanism improvements to be analyzed separately. Replay, lagged target networks, batch updates, and periodic resets may improve stability or computational efficiency in a particular training system, but they simultaneously alter how experience is used, what is stored persistently, and when updates occur. Continual-RL research must explain how these mechanisms affect later plasticity, knowledge retention, and online adaptation. Training throughput or final return alone cannot establish that the learning mechanism has improved.

9.2 Fast, Medium, and Slow Learning Processes

Multi-timescale meta-learning includes at least

- a **fast timescale**, updating state, predictions, policy, and transition models after each real event;
- a **medium timescale**, gradually regulating step sizes, credit horizons, normalization, noise suppression, and credit assignment; and
- a **slow timescale**, generating or pruning features, predictive questions, subproblems, skills, and models, while reallocating finite resources.

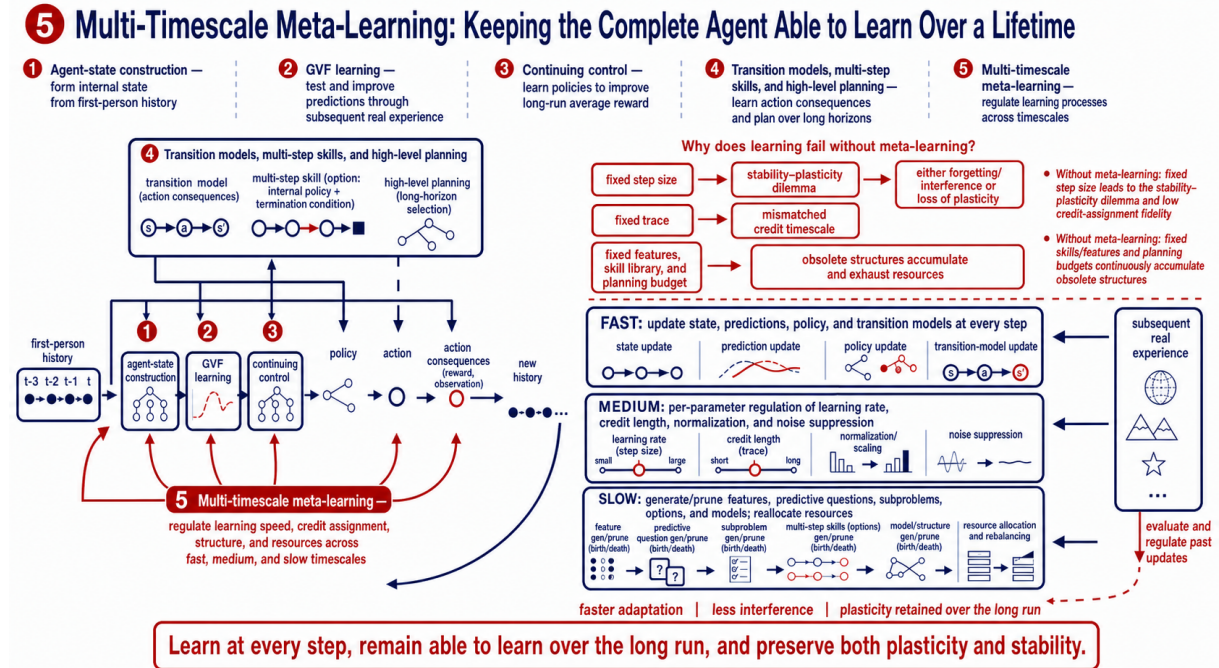


Figure 13 | Multi-timescale meta-learning regulates the first four learning processes. A fixed step size creates a long-run stability–plasticity dilemma; a fixed credit horizon creates timescale mismatch; and fixed features, skill libraries, and planning budgets accumulate obsolete structure. The goal is a testable regulation mechanism balancing rapid adaptation, low interference, and long-run plasticity.

Meta-learning is not a hyperparameter optimizer appended outside the complete agent. It must read the real consequences of the first four learning processes. Did a feature update improve a named prediction and control outcome? Did a skill reduce planning cost? Did a step-size adjustment shorten recovery after a late change? Only when these causal roles are tested is meta-learning improving continual learning rather than tuning a system from outside its lifetime.

10 Embodied Intelligence: Scaled Pretraining Cannot Replace Continual Learning in the Physical Big World

Recent robot foundation-model programs have clear value. Open X-Embodiment aggregates data across institutions and robot platforms, and RT-X demonstrates positive transfer from multi-platform data. π_0 combines a pretrained vision–language model with a flow-matching action expert to learn a general policy across multiple robots and dexterous-manipulation tasks.[Open X-Embodiment Collaboration [26]][27]

SMARTS for autonomous driving similarly shows that a designed multi-agent simulation environment can systematically expand the diversity of road-user behavior and interaction scenarios, providing important conditions for training, mechanistic analysis, and reproducible evaluation.[28] Such a platform remains an experimental environment inside the triadic structure: scene-generation rules, visible variables, task boundaries, and evaluation metrics are designer-specified. It increases interaction complexity under controlled conditions, but cannot replace the continual formation of states, objectives, and temporal abstractions from the first-person experience of a deployed agent in the physical big world.

Research on open embodied environments is moving from fixed task scores toward capability structure and combinations of unfamiliar partners and environments. Embodied Arena organizes embodied question answering, navigation, task planning, and related benchmarks into a unified capability taxonomy and evolving evaluation platform. It enables models to be compared by perceptual, reasoning, and task-execution capabilities, demonstrating that embodied competence must be analyzed across multiple environment classes and expanding data rather than summarized by the success rate of one task.[29]

Open-ended multi-robot learning further brings **unseen interaction partners** into physical systems. HOLA-Drone continually adjusts training objectives according to multi-drone cooperative relationships, studies zero-shot collaboration with unseen teammates, and validates the approach on physical drones.[30] Multi-Robot Open Adaptive Teaming simultaneously varies environments, partners, and team scales, progressively expands diversity during training, and tests cross-combination coordination on aerial and quadrupedal robots.[31] These results show that open-ended curricula, relational modeling, and physical validation can improve adaptation to unseen combinations. Their principal objectives remain post-training transfer and zero-shot coordination, rather than continual runtime modification of state, models, and skills by the same deployed agent.

Large-scale data, simulation, and foundation models can supply object semantics, action priors, and cross-task reuse. But pretraining addresses **the formation of priors from previously collected data before deployment, not the continual assimilation of self-generated experience by the same actor after deployment.**

The physical big world creates a distinctive gap between the two:

- visually similar situations can differ in friction, payload, internal faults, and contact state;
- camera, force-sensor, and joint calibration drifts, while mechanical structure wears;
- contact dynamics are discontinuous, and small deviations can produce irreversible consequences;
- the data distribution is altered by the robot’s actions, while failures cannot be collected safely and without bound;
- reward is sparse and must often be interpreted together with safety, energy, equipment lifetime, and human constraints; and
- simulation and offline data cannot enumerate new objects, materials, people, and institutions that appear after deployment.

A fleet that uploads data to the cloud, retrains several weeks later, and receives a fleet-wide update has a system-level iteration capability. If an individual deployed agent cannot calibrate sensors, revise its action model, form recovery skills, or preserve late-stage plasticity between those updates, it does not possess runtime continual-learning capability.

Learning in the physical big world must connect pretrained priors to online agent-state construction, action-conditioned models, uncertainty calibration, reward-respecting learning objectives, plasticity under bounded computation, and safe recovery.

The difficulty of embodied intelligence is not only that “the dataset is not yet large enough.” Experience is expensive and action-generated, consequences can be irreversible, state remains

hidden, and learning itself must occur safely inside the control loop. Scaled pretraining can reduce initial error, but it cannot convert action-induced distribution shifts, hardware aging, and post-deployment regularities into training data in advance. Simulation can enlarge coverage, but it cannot provide a complete generative model of real contact, sensor drift, or institutional change.

Evaluation of embodied continual learning should therefore not stop at “final success rate improved after online fine-tuning.” It should examine whether the same system forms new state features, calibrates action and option models, recovers after environmental change, and preserves existing skills and safety constraints throughout long-run deployment.

11 Model-Based Continual Reinforcement Learning in Large Language Environments

The environment of a coding, browser, or research agent is not merely its textual context. Treating the prompt or token sequence as the state, and generated text as the action, hides the interfaces that actually determine consequences.

A large language environment (LLE) contains at least five nested layers:

1. **External world:** web pages, code, files, APIs, users, organizational rules, and other actors;
2. **Runtime:** tools, permissions, long-term memory, sandboxes, model versions, and resource budgets;
3. **Internal task environment:** plans, reasoning processes, local hypotheses, and candidate models;
4. **Evidence:** tests, execution traces, verifier outputs, and state changes that actually occur in the world;
5. **Update carriers and routes:** whether an experience changes context, memory, skills, data, adapters, or model weights, and when the change is retired or rolled back.

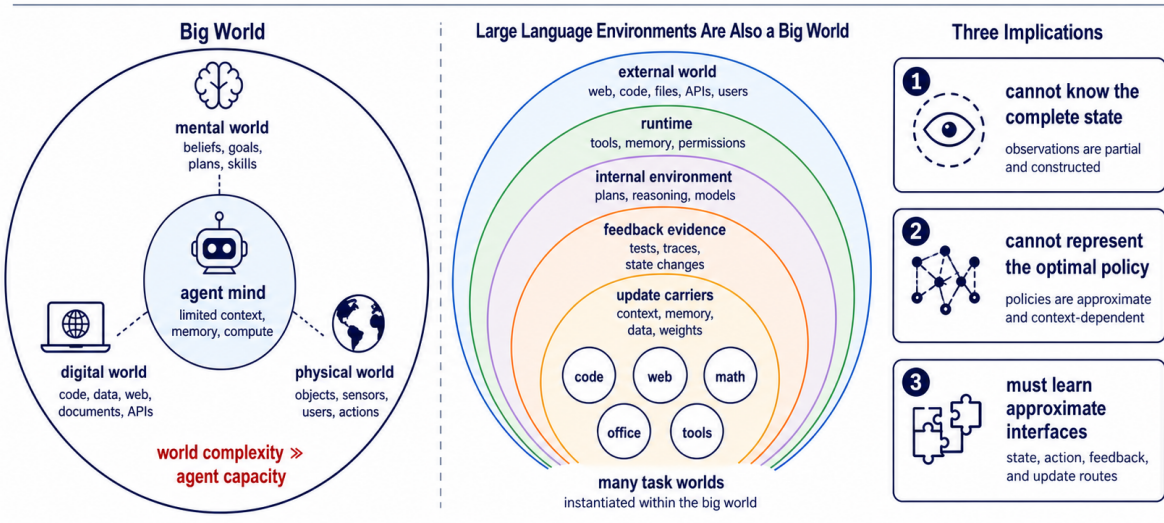
Existing reinforcement-learning work on language agents has already exposed several concrete instances of these learning problems. LLaMAC uses a modular actor–critic structure to organize internal and external feedback for large-scale multi-agent decision making, indicating that language generation, evaluation, and environmental feedback must play distinct roles within the control loop.[32] POAD observes that a single environmental action produced by a language agent is itself a structured object composed of multiple tokens. Its action decomposition addresses credit assignment both within an action and across actions, instead of treating the entire generated sequence as an indivisible atomic action.[33]

Command-line agents bring the same issue into real file systems. *Learning CLI Agents with Structured Action Credit under Selective Observation* studies both selective observation in large codebases and the assignment of sparse terminal reward to structured shell actions along long trajectories.[34] This directly reflects the central difficulty of a large language world: the agent cannot read the complete environment state at once, while its actions change files, processes, and the information that will subsequently become visible. State selection, action structure, and temporal credit therefore cannot be defined entirely inside the token-level language model.

ReMA separates large-model reasoning into a high-level meta-thinking agent, responsible for policy supervision and planning, and a low-level reasoning agent, responsible for execution; multi-agent reinforcement learning optimizes their coordination.[35] The result shows that explicit hierarchical roles can improve problem solving and generalization on specified reasoning tasks. Those roles, training objectives, and benchmark tasks are nevertheless predefined by the system. The work does not yet address how one long-running agent might discover subprob-

lems from first-person experience, construct options, and continually evaluate their long-run utility.

A Big-World View of Large Language Environments



Reinforcement learning in large language environments should seek frugal methods for learning useful interfaces in an unbounded world.

Figure 14 | The external world, runtime, internal environment, evidence, and update layers jointly constitute the environment of a language agent. Because the complete state cannot be known and an optimal policy cannot be represented exactly, the objective is not to exhaustively model the world, but to form useful state, action, feedback, and update interfaces for many task worlds under bounded resources.

Experience memory provides another learnable update carrier. DS-Agent uses case-based reasoning to organize iterative data-science agents and reuses prior cases for later planning and code execution, demonstrating how successful experience can be structured and invoked on new problems.[36] MemRL goes further by applying runtime reinforcement learning to episodic memory while keeping the language model frozen: it first retrieves semantically relevant candidate experiences and then uses Q-values, updated from environmental feedback, to estimate their utility.[37] Such work makes *what to remember, when to retrieve it, and whether it was useful* learnable objects rather than simply appending similar trajectories to context. A complete continual-RL system must further connect memory selection to agent-state construction, continuing control, model calibration, knowledge retirement, and autonomous learning-problem construction.

This distinction changes the answer to “what should a world model model?” In a large language world, the objective should not be to simulate the entire internet. The agent should learn action-relevant interface models that can be falsified by subsequent events, for example:

- under the current permissions and software versions, what a tool call will return, how long it will take, and how it may fail;
- under what task conditions a memory remains applicable, and whether retrieving it reduces real interaction and verification costs;
- which files, tests, and future observations a modification will affect, and how the change can be rolled back;
- where a retrieval, verification, or recovery option terminates, and its probability of success, duration, cost, and risk;
- which internal predictions are actually read by the policy or planner and change root-action selection.

A log is not automatically experience, and a tool response is not automatically reward. A zero exit status, a passing test, or positive user feedback is only local evidence. Whether it indicates progress toward the long-run objective depends on the preceding state, the action actually executed, the resulting causal consequences, and independent verification. Writing a successful trajectory to memory is not by itself continual learning: the record may have become obsolete, or may apply only to another permission regime, version, or task phase.

Experience in a large language world therefore cannot mean “store more trajectories.” A reusable experience object should at least identify what the system knew, what it did, what actually changed in the world, what evidence verified the change, where the update was written, and under what future conditions it may be reused or revoked. Only updates that are executable, verifiable, reversible, and explicit about their domain of applicability can accumulate utility under bounded resources.

Experience Flywheel: Learn in Small World, Generalize to Big World

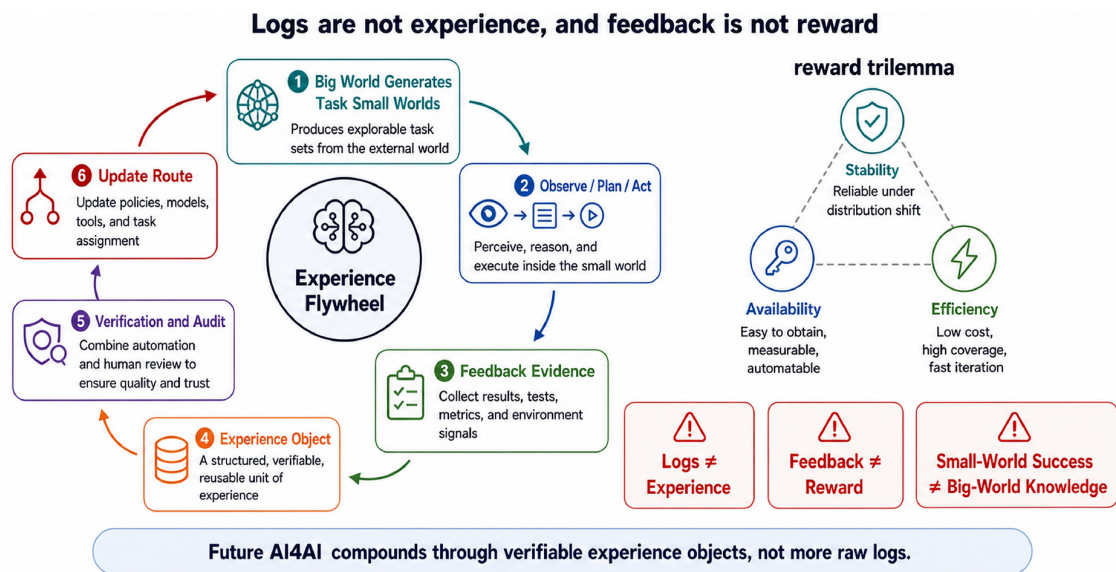


Figure 15 | The big world generates testable small task worlds. The agent observes, plans, acts, and collects feedback; experience enters an update route only after it has been structured, verified, and audited. Logs are not experience, feedback is not reward, and local success is not knowledge of the big world. These distinctions determine whether experience can be reused over the long run.

This perspective connects model-based learning, continual learning, and agent systems as one problem. Models form reusable action-conditional knowledge; continual learning keeps that knowledge aligned with a changing world and policy; verification procedures and update criteria determine whether knowledge may safely influence future behavior. Related discussions appear in *What Environment Do LLM Agents Learn In?* and *What Is a World Model Modeling? From Predicting the Future to Reusing Experience*.

12 Empirical Research on Continual Reinforcement Learning

The object of evaluation in continual reinforcement learning is how the same agent changes during uninterrupted interaction, not merely a fixed parameter vector obtained at the end of training. The object of study includes both external behavior and the learning processes that produce it: how the agent constructs state, updates predictions and control, revises transition models, constructs internal learning problems, and regulates these processes over long opera-

tion. Adaptation, retention, recovery, and resource use across a complete run jointly characterize continual-learning ability.

12.1 Objects of Evaluation and Principles for Controlled Comparison

Experiments should cover the continuous interaction process from initialization onward. Repeated resets, task labels, or future information available only during evaluation should not substitute for the agent’s own learning. The true environmental state, change points, and task boundaries may be used to construct oracle controls and diagnose sources of error, but should not be supplied to the evaluated agent unless the method explicitly assumes them. Comparisons should also place real interaction, experience reuse, persistent storage, planning, and computation under comparable conditions.

Average reward or cumulative return during continual interaction remains an important control outcome, but a single behavioral measure cannot identify the mechanism responsible for improvement. The same return difference may arise from more informative agent state, more stable streaming updates, broader exploration coverage, greater late-stage plasticity, or more useful internal learning problems. Complete environments are needed to test integrated behavior; controlled environments and component comparisons are needed to determine what state construction, prediction, control, models, objective construction, and meta-learning each contribute.

12.2 Central Research Directions in Continual Reinforcement Learning

Continual reinforcement learning is not captured by one network architecture or one isolated algorithmic problem. Following the computational dependencies developed above, several connected research directions are central.

Streaming prediction and control. Base learners must update incrementally from time-ordered experience generated by the current policy, while handling bootstrapping, off-policy learning, long-term credit assignment, and changing data distributions under bounded computation and storage. Whether a deep network can remain numerically stable and capable of learning over long periods without frequent resets or large-scale offline retraining is a prerequisite for its inclusion in a complete continual agent.

Continual construction of agent state. From histories of partial observation, an agent must discover distinctions that are useful for future prediction and action, then revise its internal state as sensors, embodiment, policy, and environment change. Central problems include generating and retiring state features, keeping changes in shared representation consistent with values, policies, models, and learning state, and preventing bounded memory from persistently aliasing histories with different consequences.

Knowledge retention and long-run plasticity. A continual learner must preserve predictions, skills, and models that remain applicable while still acquiring new regularities late in its lifetime. Catastrophic forgetting and loss of plasticity are distinct: the former asks whether established functions are damaged, whereas the latter asks whether the system can still form new functions. Research is needed on capacity allocation, structural change, selective reuse and forgetting, and their long-run balance under fixed resources.

General value functions, transition models, and temporal abstraction. An agent must learn GVF’s defined by different target policies, cumulants, timescales, and termination conditions, together with action models and option models useful for control and planning. The division of labor among general value prediction, one-step models, multi-step models, and temporal abstraction; the control of model error in compositional planning; and the requirement that pre-

dictions demonstrably improve action selection are central problems for model-based continual reinforcement learning.

Autonomous learning problems and objective construction. The number of possible predictions, subtasks, and options in a big world far exceeds an agent’s capacity. A learner must generate candidate problems from experience; estimate their learnability, downstream use, and long-run utility; control their exploration, storage, and interference costs; and retire structures that have ceased to be useful. Autonomous objective construction is not the addition of arbitrary intrinsic rewards. It is the construction of internal learning problems that support long-run control while remaining consistent with the external reward.

Regulation across multiple timescales. State, predictions, control, models, and skills change at different rates. A single fixed step size, credit-assignment horizon, or update frequency is unlikely to suit a complete lifetime. Meta-learning must use actual predictive and control consequences to regulate learning rates, credit assignment, structural generation, and resource allocation, producing a sustainable relationship among rapid adaptation, knowledge retention, and long-run plasticity.

These directions cannot be completed independently. State determines which predictions are learnable; control determines which experience is observed; predictions and models support skills and planning; autonomous objectives determine how bounded capacity is used; and meta-learning regulates how all these objects change over time. Empirical continual-RL research must ultimately establish whether these mechanisms, operating jointly in one learner over one stream of experience, continually improve behavior and form new useful knowledge.

13 Conclusion: Continual Reinforcement Learning and the Formation of Intelligence

Continual reinforcement learning requires a complete causal chain of research:

- direct interaction with a big world
 - first-person history
 - agent-state construction
 - general value-function learning
 - continuing control
 - transition models, autonomous objectives, and planning
 - multi-timescale meta-learning
 - continual revision of the learner.

Bandits isolate exploration and feedback; MDPs provide a theoretical structure for state, time, and return; tabular methods expose the update semantics of learning; and deep networks extend function approximation to high-dimensional perception and complex control. Each addresses essential parts of this chain. None automatically determines which states should be formed, which predictions are worth learning, which skills should be retained, or how to evaluate a learning process that continually changes itself.

The research program of continual reinforcement learning is to return objects that have been fixed by the environment or engineering system to the domain of learning, while preserving clear and testable functional relationships among them:

1. use triadic experiments to isolate mechanisms, while progressively reducing the state, tasks, and abstractions supplied by the designer;

2. construct revisable agent state from first-person history in dyadic interaction with a big world;
3. make predictions falsifiable knowledge with an explicit policy, timescale, and domain of applicability;
4. use models, options, and planning to reuse experience rather than to construct a complete replica of the world;
5. determine learning objectives through candidate generation, downstream use, tests against real consequences, and selection and retirement under fixed capacity;
6. enable the same bounded learner to retain plasticity, stability, recovery ability, and controlled resource use over long operation;
7. evaluate the formation of intelligence through behavior, adaptation, recovery, risk, and cost across complete runs, rather than through peak performance at the training endpoint.

Modern deep reinforcement learning is not the opposite of this program. It supplies an important layer of solution methods already established within it. Continual reinforcement learning changes the fundamental problem: from *how to find a better policy in a given environment model or problem specification* to *how a bounded actor, in an inexhaustible world, continually forms, tests, and revises its own state, predictions, control, models, learning objectives, and learning mechanisms*. This remains a central unfinished problem for reinforcement learning as a science of intelligence.

For broader definitions and perspectives on continual reinforcement learning, see Khimya Khetarpal et al., *Towards Continual Reinforcement Learning: A Review and Perspectives* [38], JAIR 2022; David Abel et al., *A Definition of Continual Reinforcement Learning* [39], NeurIPS 2023; and Richard S. Sutton, Michael Bowling, and Patrick M. Pilarski, *The Alberta Plan for AI Research* [3], 2022. For an introduction organized around the classical family of reinforcement-learning algorithms, see Lilian Weng’s *A (Long) Peek into Reinforcement Learning* [40].

References

- [1] David Silver, Satinder Singh, Doina Precup, and Richard S. Sutton. Reward Is Enough. *Artificial Intelligence*, 299:103535, 2021. doi: 10.1016/j.artint.2021.103535. URL <https://doi.org/10.1016/j.artint.2021.103535>.
- [2] Openmind Research Institute. The Question of How Minds Can Form Is at the Core of AI. Institutional research statement, 2026. URL <https://www.openmindresearch.org/>. Accessed 2026-09-06.
- [3] Richard S. Sutton, Michael Bowling, and Patrick M. Pilarski. The Alberta Plan for AI Research. arXiv preprint arXiv:2208.11173, 2022. URL <https://arxiv.org/abs/2208.11173>.
- [4] Richard S. Sutton. RLSS 2026: Reinforcement Learning Course Materials. Lecture slides and course materials, 2026. URL <https://drive.google.com/drive/folders/1wtHmiM5mKKLg14K4XCUGeKdDVXAOLD9P>. Includes Bitter Lesson and Big World, Control with Function Approximation and Average Reward, Off-policy Learning and the Deadly Triad, OaK, and GVFs and Subproblems.
- [5] Richard S. Sutton. The Bitter Lesson. Online essay, 2019. URL <http://www.incompleteideas.net/IncIdeas/BitterLesson.html>.
- [6] Khurram Javed and Richard S. Sutton. The Big World Hypothesis and Its Ramifications for Artificial Intelligence. Reinforcement Learning Conference workshop position paper, 2024. URL <https://openreview.net/forum?id=Sv7DazuCn8>.
- [7] Kris De Asis. Design for Learning. Online essay, 2026. URL <https://kris.pengy.ca/designforlearning>.
- [8] Richard S. Sutton. Hidden State: What It Is, What to Do about It. RLSS 2026 lecture material, 2026. URL <https://drive.google.com/drive/folders/1wtHmiM5mKKLg14K4XCUGeKdDVXAOLD9P>.
- [9] David Silver and Richard S. Sutton. Welcome to the Era of Experience. Preprint chapter, 2025. URL <https://storage.googleapis.com/deepmind-media/Era-of-Experience%20/The%20Era%20of%20Experience%20Paper.pdf>.

- [10] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018. URL <http://incompleteideas.net/book/RLbook2020.pdf>.
- [11] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Belle-mare, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. doi: 10.1038/nature14236. URL <https://www.nature.com/articles/nature14236>.
- [12] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, et al. Mastering the game of Go without human knowledge. *Nature*, 550(7676):354–359, 2017. doi: 10.1038/nature24270. URL <https://www.nature.com/articles/nature24270>.
- [13] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, et al. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604–609, 2020. doi: 10.1038/s41586-020-03051-4. URL <https://www.nature.com/articles/s41586-020-03051-4>.
- [14] Muning Wen, Jakub Grudzien Kuba, Runji Lin, Weinan Zhang, Ying Wen, Jun Wang, and Yaodong Yang. Multi-Agent Reinforcement Learning is a Sequence Modeling Problem. arXiv preprint arXiv:2205.14953, 2022. URL <https://arxiv.org/abs/2205.14953>.
- [15] Richard S. Sutton. The Quest for a Common Model of the Intelligent Decision Maker. arXiv preprint arXiv:2202.13252, 2022. URL <https://arxiv.org/abs/2202.13252>.
- [16] Richard S. Sutton, Joseph Modayil, Michael Delp, Thomas Degris, Patrick M. Pilarski, Adam White, and Doina Precup. Horde: A Scalable Real-time Architecture for Learning Knowledge from Unsupervised Sensorimotor Interaction. In *Proceedings of the 10th International Conference on Autonomous Agents and Multiagent Systems*, pages 761–768, 2011. URL https://sites.ualberta.ca/~pilarski/docs/papers/Sutton_2011_Horde_AAMAS.pdf.
- [17] Abhishek Naik, Roshan Shariff, Niko Yasui, Hengshuai Yao, and Richard S. Sutton. Discounted Reinforcement Learning Is Not an Optimization Problem. arXiv preprint arXiv:1910.02140, 2019. URL <https://arxiv.org/abs/1910.02140>.
- [18] Xidong Feng, Oliver Slumbers, Ziyu Wan, Bo Liu, Stephen McAleer, Ying Wen, Jun Wang, and Yaodong Yang. Neural Auto-Curricula. arXiv preprint arXiv:2106.02745, 2021. URL <https://arxiv.org/abs/2106.02745>.
- [19] Nicolas Perez Nieves, Yaodong Yang, Oliver Slumbers, David Henry Mguni, Ying Wen, and Jun Wang. Modelling Behavioural Diversity for Learning in Open-Ended Games. arXiv preprint arXiv:2103.07927, 2021. URL <https://arxiv.org/abs/2103.07927>.
- [20] Yang Li, Shao Zhang, Jichen Sun, Yali Du, Ying Wen, Xinbing Wang, and Wei Pan. Cooperative Open-ended Learning Framework for Zero-shot Coordination. arXiv preprint arXiv:2302.04831, 2023. URL <https://arxiv.org/abs/2302.04831>.
- [21] Ming Zhou, Ziyu Wan, Hanjing Wang, Muning Wen, Runzhe Wu, Ying Wen, et al. MALib: A Parallel Framework for Population-based Multi-agent Reinforcement Learning. arXiv preprint arXiv:2106.07551, 2021. URL <https://arxiv.org/abs/2106.07551>.
- [22] Richard S. Sutton, Doina Precup, and Satinder Singh. Between MDPs and Semi-MDPs: A Framework for Temporal Abstraction in Reinforcement Learning. *Artificial Intelligence*, 112(1–2):181–211, 1999. doi: 10.1016/S0004-3702(99)00052-1. URL [https://doi.org/10.1016/S0004-3702\(99\)00052-1](https://doi.org/10.1016/S0004-3702(99)00052-1).
- [23] Mohamed Elsayed, Gautham Vasan, and A. Rupam Mahmood. Streaming Deep Reinforcement Learning Finally Works. arXiv preprint arXiv:2410.14606, 2024. URL <https://arxiv.org/abs/2410.14606>.
- [24] Shibhansh Dohare, J. Fernando Hernandez-Garcia, Qingfeng Lan, Parash Rahman, A. Rupam Mahmood, and Richard S. Sutton. Loss of plasticity in deep continual learning. *Nature*, 632(8026):768–774, 2024. doi: 10.1038/s41586-024-07711-7. URL <https://www.nature.com/articles/s41586-024-07711-7>.
- [25] Shibhansh Dohare, Richard S. Sutton, and A. Rupam Mahmood. Continual Backprop: Stochastic Gradient Descent with Persistent Randomness. arXiv preprint arXiv:2108.06325, 2021. URL <https://arxiv.org/abs/2108.06325>.

- [26] Open X-Embodiment Collaboration. Open X-Embodiment: Robotic Learning Datasets and RT-X Models. arXiv preprint arXiv:2310.08864, 2023. URL <https://arxiv.org/abs/2310.08864>.
- [27] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, et al. π_0 : A Vision-Language-Action Flow Model for General Robot Control. arXiv preprint arXiv:2410.24164, 2024. URL <https://arxiv.org/abs/2410.24164>.
- [28] Ming Zhou, Jun Luo, Julian Vilella, Yaodong Yang, David Rusu, Jiayu Miao, et al. SMARTS: Scalable Multi-Agent Reinforcement Learning Training School for Autonomous Driving. arXiv preprint arXiv:2010.09776, 2020. URL <https://arxiv.org/abs/2010.09776>.
- [29] Fei Ni, Min Zhang, Pengyi Li, Yifu Yuan, Lingfeng Zhang, Yuecheng Liu, et al. Embodied Arena: A Comprehensive, Unified, and Evolving Evaluation Platform for Embodied AI. arXiv preprint arXiv:2509.15273, 2025. URL <https://arxiv.org/abs/2509.15273>.
- [30] Yang Li, Dengyu Zhang, Junfan Chen, Ying Wen, Qingrui Zhang, Shaoshuai Mou, and Wei Pan. HOLA-Drone: Hypergraphic Open-ended Learning for Zero-Shot Multi-Drone Cooperative Pursuit. arXiv preprint arXiv:2409.08767, 2024. URL <https://arxiv.org/abs/2409.08767>.
- [31] Yang Li, Feng Xue, Fan Mo, Yunhao Liu, Jianhong Wang, Ying Wen, et al. Multi-Robot Open Adaptive Teaming Across Unseen Environments, Partners, and Scales. arXiv preprint arXiv:2607.04972, 2026. URL <https://arxiv.org/abs/2607.04972>.
- [32] Bin Zhang, Hangyu Mao, Jingqing Ruan, Ying Wen, Yang Li, Shao Zhang, et al. Controlling Large Language Model-based Agents for Large-Scale Decision-Making: An Actor-Critic Approach. arXiv preprint arXiv:2311.13884, 2023. URL <https://arxiv.org/abs/2311.13884>.
- [33] Muning Wen, Ziyu Wan, Weinan Zhang, Jun Wang, and Ying Wen. Reinforcing Language Agents via Policy Optimization with Action Decomposition. arXiv preprint arXiv:2405.15821, 2024. URL <https://arxiv.org/abs/2405.15821>.
- [34] Haoyang Su and Ying Wen. Learning CLI Agents with Structured Action Credit under Selective Observation. arXiv preprint arXiv:2605.08013, 2026. URL <https://arxiv.org/abs/2605.08013>.
- [35] Ziyu Wan, Yunxiang Li, Xiaoyu Wen, Yan Song, Hanjing Wang, Linyi Yang, et al. ReMA: Learning to Meta-think for LLMs with Multi-Agent Reinforcement Learning. arXiv preprint arXiv:2503.09501, 2025. URL <https://arxiv.org/abs/2503.09501>.
- [36] Siyuan Guo, Cheng Deng, Ying Wen, Hechang Chen, Yi Chang, and Jun Wang. DS-Agent: Automated Data Science by Empowering Large Language Models with Case-Based Reasoning. arXiv preprint arXiv:2402.17453, 2024. URL <https://arxiv.org/abs/2402.17453>.
- [37] Shengtao Zhang, Jiaqian Wang, Ruiwen Zhou, Junwei Liao, Yuchen Feng, Zhuo Li, et al. MemRL: Self-Evolving Agents via Runtime Reinforcement Learning on Episodic Memory. arXiv preprint arXiv:2601.03192, 2026. URL <https://arxiv.org/abs/2601.03192>.
- [38] Khimya Khetarpal, Matthew Riemer, Irina Rish, and Doina Precup. Towards Continual Reinforcement Learning: A Review and Perspectives. arXiv preprint arXiv:2012.13490, 2020. URL <https://arxiv.org/abs/2012.13490>.
- [39] David Abel, André Barreto, Benjamin Van Roy, Doina Precup, Hado van Hasselt, and Satinder Singh. A Definition of Continual Reinforcement Learning. arXiv preprint arXiv:2307.11046, 2023. URL <https://arxiv.org/abs/2307.11046>.
- [40] Lilian Weng. A (Long) Peek into Reinforcement Learning. Online tutorial, 2018. URL <https://lilianweng.github.io/posts/2018-02-19-rl-overview/>.